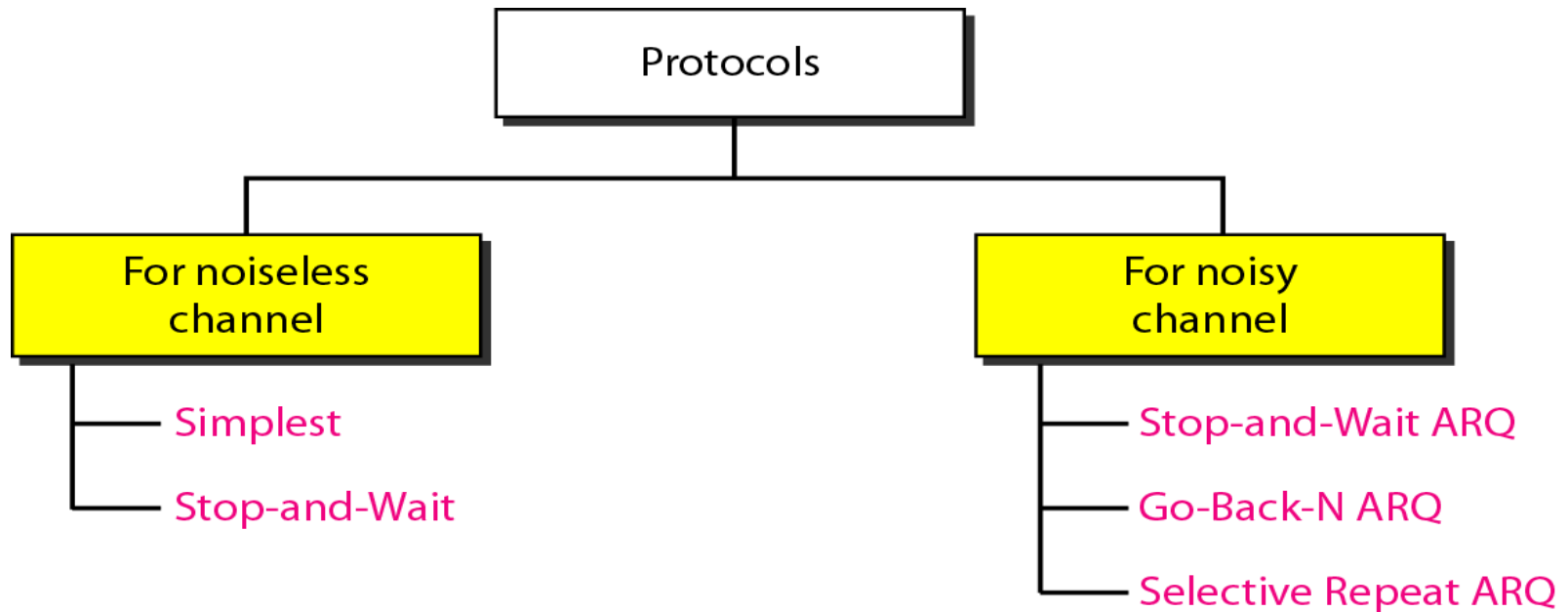


FLOW-CONTROL PROTOCOLS

Elementary Data Link Protocols

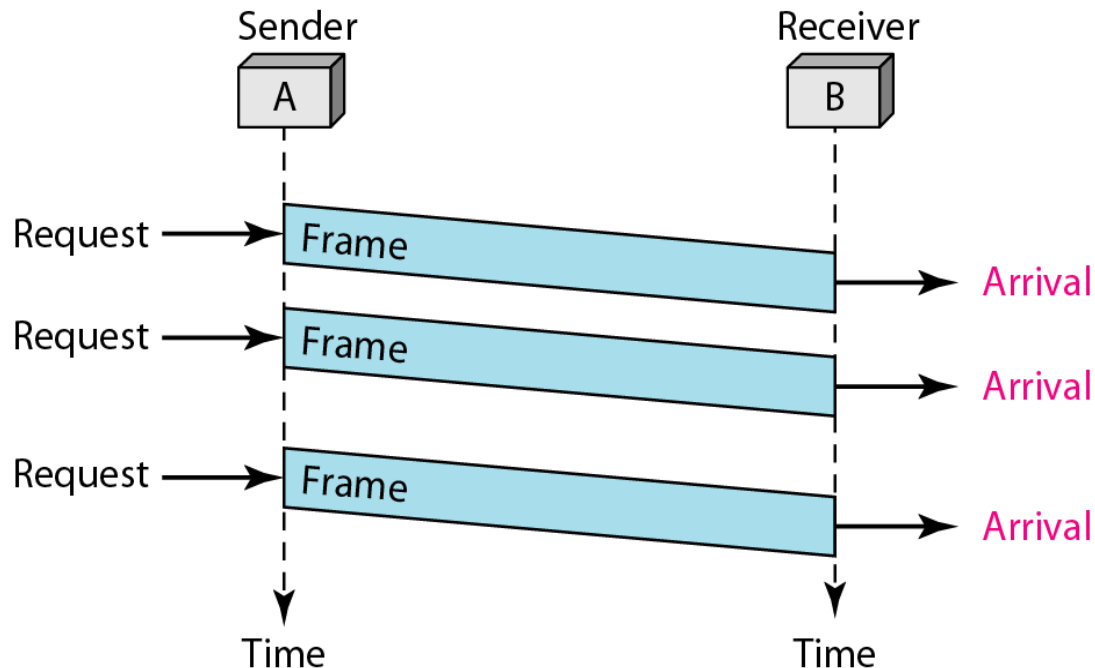


1. An Unrestricted Simplex Protocol
2. A Simplex Stop-and-Wait Protocol
3. A Simplex Protocol for a Noisy Channel

STOP AND WAIT PROTOCOLS

1. An unrestricted simplex protocol

- To appreciate the step-by-step development of efficient and complex protocols we will begin with a simple but unrealistic protocol.
- In this protocol: Data are transmitted in one direction only
- The transmitting (Tx) and receiving (Rx) hosts are always ready
- Processing time can be ignored
- Infinite buffer space is available
- No errors occur; i.e., no damaged frames and no lost frames (perfect channel)



2. A simplex stop-and-wait protocol

- In this protocol we assume that Data are transmitted in one direction only
- No errors occur (perfect channel)
- The receiver can only process the received information at a finite rate
- These assumptions imply that the transmitter cannot send frames at a rate faster than the receiver can process them.

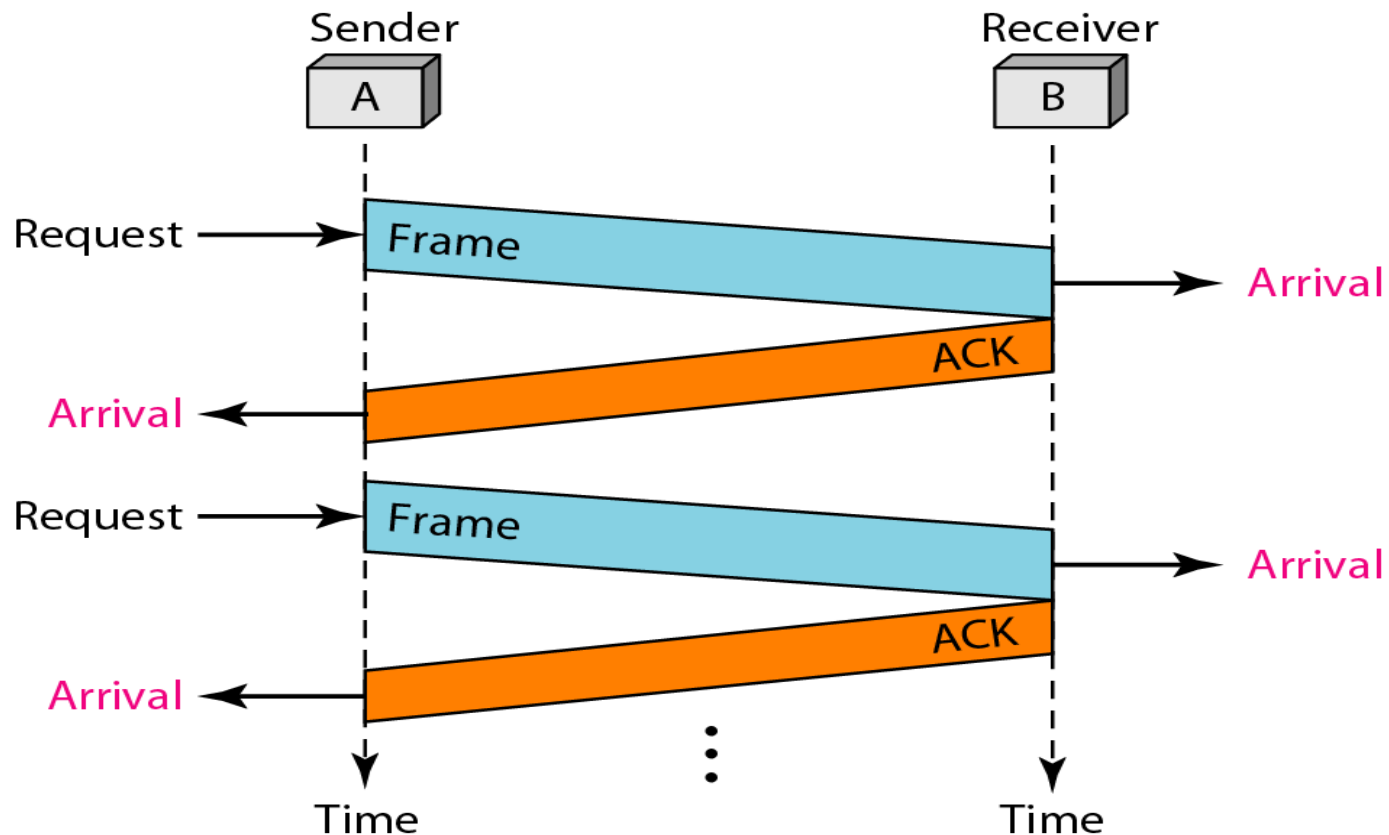
The problem here is how to prevent the sender from flooding the receiver.

- A general solution to this problem is to have the receiver provide some sort of feedback to the sender.
- The process could be as follows:
 - The receiver send an acknowledge frame back to the sender telling the sender that the last received frame has been processed and passed to the host;
 - permission to send the next frame is granted.
 - The sender, after having sent a frame, must wait for the acknowledge frame from the receiver before sending another frame.

This protocol is known as *stop-and-wait*.

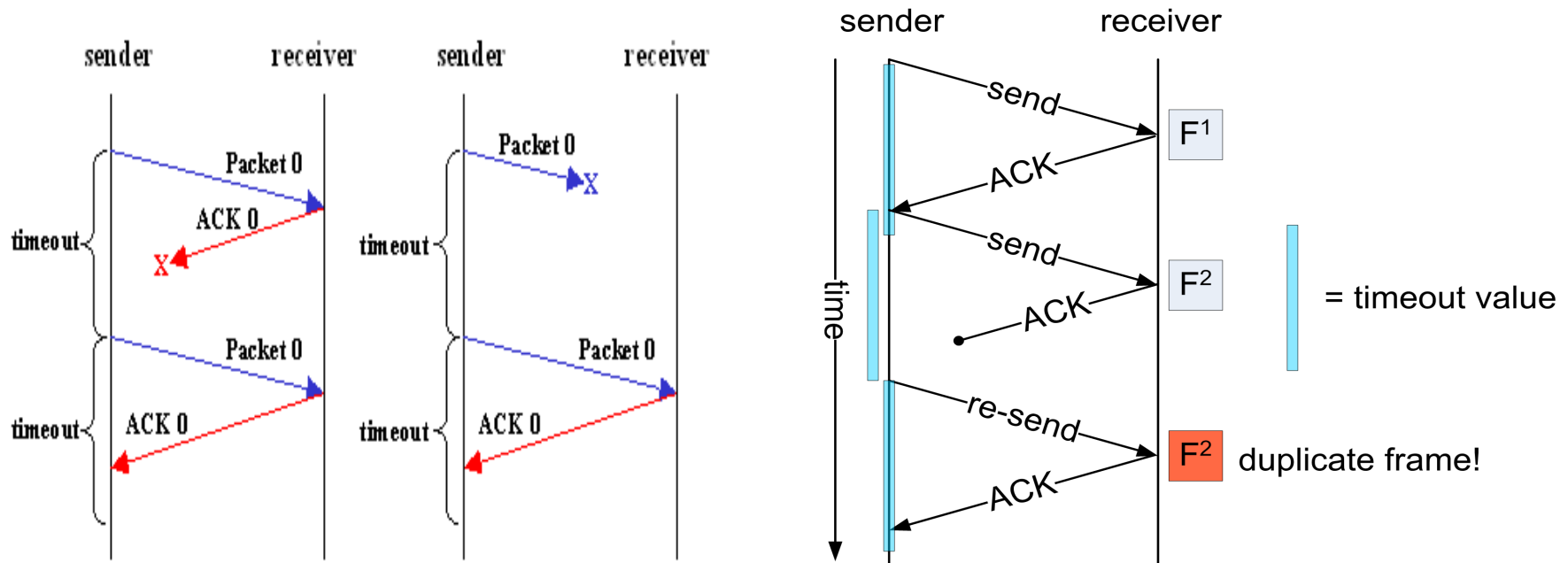
Stop & Wait Protocol

The sender sends one frame and waits for feedback from the receiver. When the ACK arrives, the sender sends the next frame



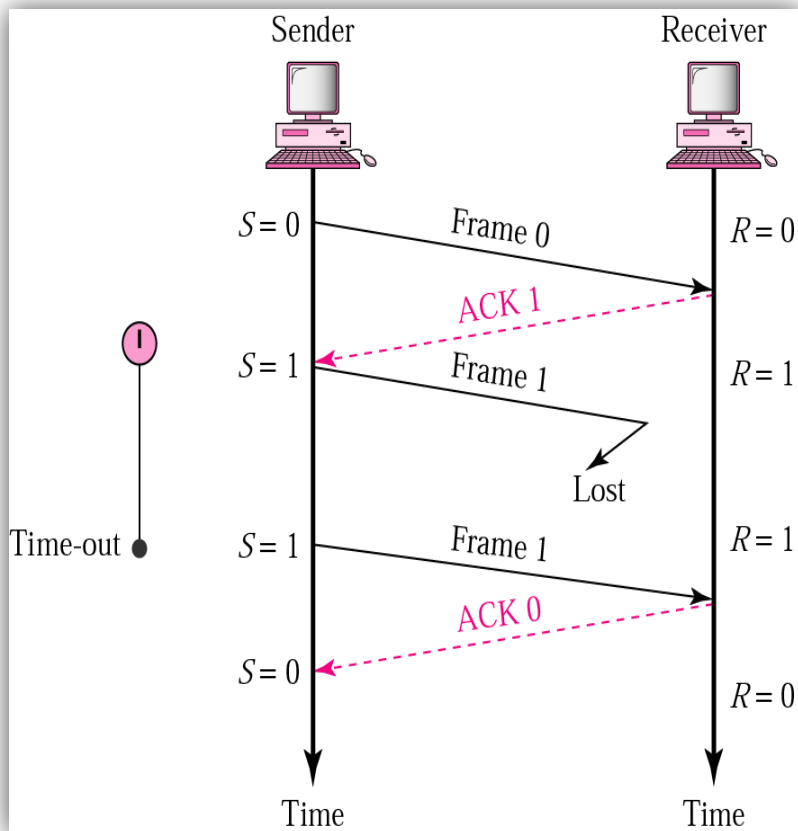
3. A Simplex Protocol for a Noisy Channel

- In this protocol the unreal "error free" assumption in protocol 2 is dropped. Frames may be either damaged or lost completely.
- We assume that transmission errors in the frame are detected by the hardware checksum.
- One suggestion is that the sender would send a frame, the receiver would send an ACK frame only if the frame is received correctly.
- If the frame is in error the receiver simply ignores it; the transmitter would time out and would retransmit it.
- One fatal flaw with the above scheme is that if the ACK frame is lost or damaged, duplicate frames are accepted at the receiver without the receiver knowing it.

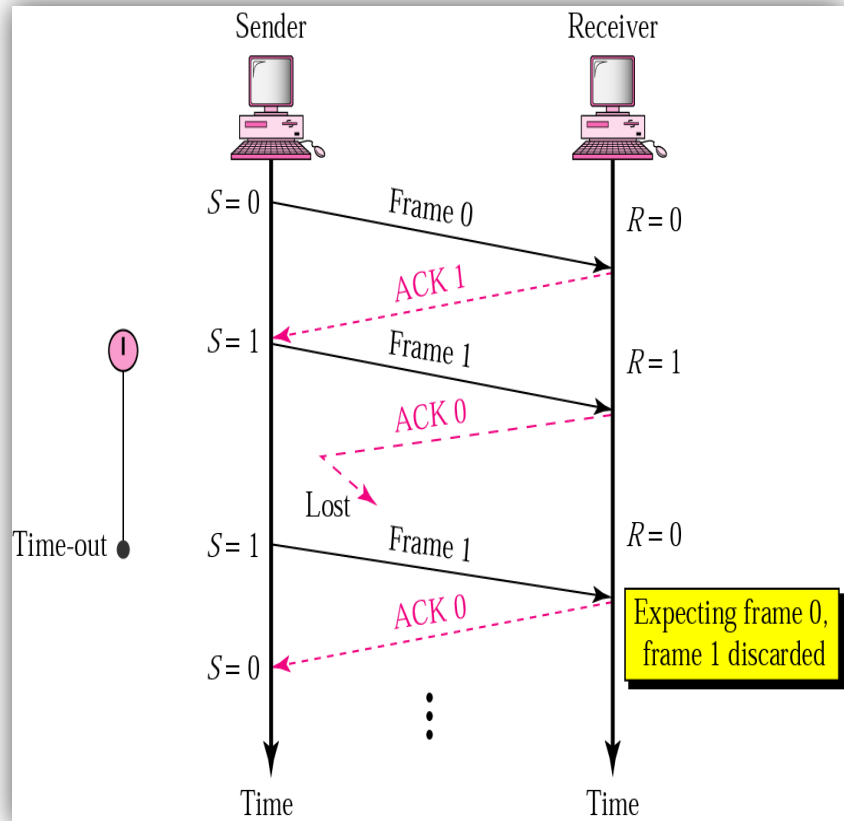


- Imagine a situation where the receiver has just sent an ACK frame back to the sender saying that it correctly received and already passed a frame to its host.
- However, the ACK frame gets lost completely, the sender times out and retransmits the frame.
- There is no way for the receiver to tell whether this frame is a retransmitted frame or a new frame, so the receiver accepts this duplicate happily and transfers it to the host. The protocol thus fails in this aspect.

STOP-AND-WAIT, LOST FRAME

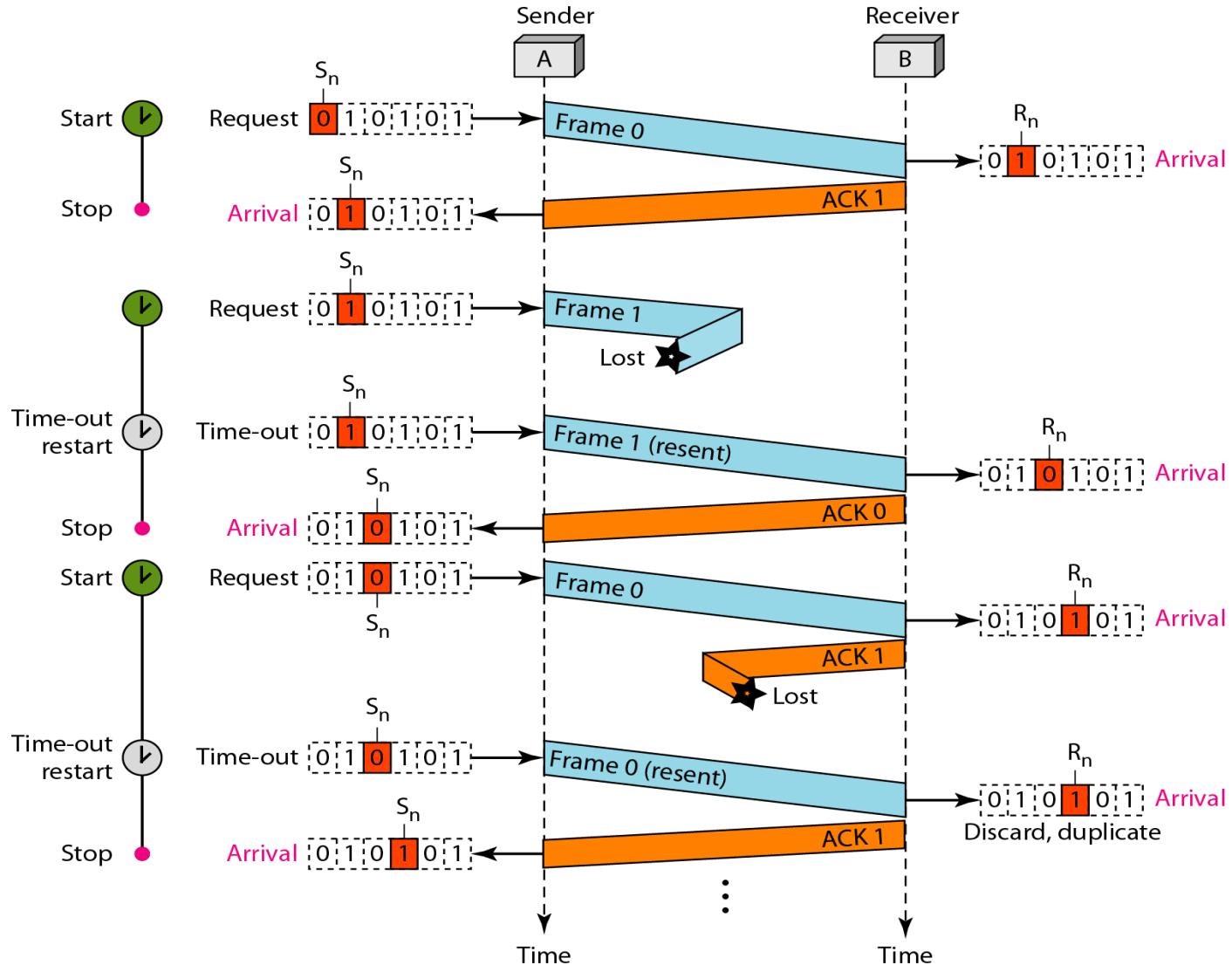


STOP-AND-WAIT, LOST ACK FRAME



- To overcome this problem, it is required that the receiver be able to distinguish a frame that it is seeing for the first time from a retransmission.
- One way to achieve this is to have the sender put a **sequence number** in the header of each frame it sends.
- The receiver then can check the sequence number of each arriving frame to see if it is a new frame or a duplicate to be discarded.
- The receiver needs to distinguish only 2 possibilities: a new frame or a duplicate; a **sequence number** is sufficient to distinguish.
- At any instant, the receiver expects a particular sequence number.
- Any **wrong sequence numbered frame** arriving at the receiver is **rejected as a duplicate**.
- A **correctly numbered frame** arriving at the receiver is accepted, passed to the host, and the **expected sequence number is incremented by 1** (modulo 2).

Flow Diagram of a Stop & Wait Protocol



- After transmitting a frame and starting the timer, the sender waits for something exciting to happen.
 - Only three possibilities exist:
 - an acknowledgement frame arrives undamaged,
 - a damaged acknowledgement frame staggers in, or
 - the timer expires.
- If a *valid acknowledgement* comes in, the sender fetches the next packet from its network layer and puts it in the buffer, overwriting the previous packet.
 - It also advances the sequence number.
- If a *damaged frame arrives or no frame at all* arrives, neither the buffer nor the sequence number is changed so that a duplicate can be sent.
- When a valid frame arrives at the receiver, its sequence number is checked to see if it is a duplicate.
 - If not, it is accepted, passed to the network layer, and an acknowledgement is generated. Duplicates and damaged frames are not passed to the network layer.

If the bandwidth of the line is 1.5 Mbps, Round Trip Time (RTT) is 45 msec and packet size is 1 KB, then find the **link utilization** in stop and wait.

Transmission Delay (T_t)-

= Packet size / Bandwidth

$$= 1 \text{ KB} / 1.5 \text{ Mbps}$$

$$= (2^{10} \times 8 \text{ bits}) / (1.5 \times 10^6 \text{ bits per sec})$$

$$= 5.461 \text{ msec}$$

Propagation Delay (T_p)-

= Round Trip Time / 2

$$= 45 \text{ msec} / 2$$

$$= 22.5 \text{ msec}$$

Value Of 'a'-

$$a = T_p / T_t$$

$$a = 22.5 \text{ msec} / 5.461 \text{ msec}$$

$$a = 4.12$$

Link Utilization/ Efficiency (η) -

$$= \frac{T_t}{T_t + 2T_p} = 1 / 1 + 2a$$

$$= 1 / (1 + 2 \times 4.12)$$

$$= 1 / 9.24$$

$$= 0.108$$

$$= 10.8 \%$$

If the packet size is 1 KB and propagation time is 15 ms, the channel capacity is 10^9 bps, then find the transmission time and utilization of sender in stop and wait protocol.

Transmission Delay (T_t)-

= Packet size / Bandwidth

= 1 KB / 10^9 bps

= 8.192 μ sec

Propagation Delay (T_p)-

= 15 ms

Value Of 'a'-

$$a = T_p / T_t$$

a = 15 ms / 8.192 μ sec

a = 1831.054

Link Utilization/ Efficiency (η) -

$$= 1 / (1 + 2a)$$

= 1 / (1 + 2 x 1831.054)

= 1 / 3662.108

= 0.000273

= 0.0273 %

Consider two hosts X and Y connected by a single direct link of rate 10^6 bits/sec. The distance between the two hosts is 10,000 km and the propagation speed along the link is 2×10^8 m/sec. Host X sends a file of 50,000 bytes as one large message to host Y continuously. Let the transmission and propagation delays be p milliseconds and q milliseconds respectively. Then the value of p and q are-

- A. $p = 50$ and $q = 100$
- B. $p = 50$ and $q = 400$
- C. $p = 100$ and $q = 50$
- D. $p = 400$ and $q = 50$

Transmission Delay (T_t)-

= Packet size / Bandwidth

= 50000 bytes / 10^6 bits
per sec

= 400 msec

- Answer = D

Propagation Delay (T_p)-

= Distance / Propagation
Speed

= 10000 km / (2 x
 10^8 m/sec)

= 50 msec

Using stop and wait protocol, sender wants to transmit 10 data packets to the receiver. Out of these 10 data packets, every 4th data packet is lost. How many packets sender will have to send in total?

Solution:

1,2,3,4,4,5,6,7,7,8,9,10,10

13 packets will be sent.

A sender uses the stop and wait ARQ protocol for reliable transmission of frames. Frames are of size 1000 bytes and the transmission rate at the sender is 80 Kbps. Size of an acknowledgement is 100 bytes and the transmission rate at the receiver is 8 Kbps. The one way propagation delay is 100 msec.

Assuming no frame is lost, the sender throughput is _____ bytes/sec.

Transmission Delay (T_t)-

Frame size / Sender bandwidth

= 1000 bytes / 80 Kbps

= 100 msec

Transmission delay (T_t)-

= Acknowledgement size / Receiver bandwidth

= 100 bytes / 8 Kbps

= (100 x 8 bits) / (8 x 10³ bits per sec)

= 100 msec

Propagation Delay (T_p)-

= 100 ms

Useful Time-

= Transmission delay of data frame

= 100 msec

Total Time-

= Transmission delay of data frame +

Propagation delay of data frame +

Transmission delay of

acknowledgement + Propagation delay of acknowledgement

= 100 msec + 100 msec + 100 msec +

100 msec

= 400 msec

Efficiency (η)-

= $\frac{T_t}{2(T_t + T_p)}$

= Useful time / Total time

= 100 msec / 400 msec

= 25% = 0.25

Sender Throughput-

= Efficiency (η) x Sender bandwidth

= 0.25 x 80 Kbps

= 20 Kbps = (20 x 1000 / 8) bytes per sec

= 2500 bytes/sec

SLIDING WINDOW PROTOCOLS

Data Frame Transmission

- **Unidirectional assumption** in previous elementary protocols

⇒ **Not general**

- **Full-duplex** - approach **1**

- Two separate communication channels(physical circuits)
 - Forward channel for data
 - Reverse channel for acknowledgement

⇒ **Problems**: 1. reverse channel bandwidth wasted
2. cost

Data Frame Transmission

- **Full-duplex** - approach 2
 - Same circuit for both directions
 - **Data and acknowledgement are intermixed**
 - How do we tell acknowledgement from data?
"kind" field telling data or acknowledgement
 - Can it be improved?
- **Full-duplex** - Approach 3
 - Attaching acknowledgement to outgoing data frames
⇒ **PIGGYBACKING**

Piggybacking

- Temporarily delaying transmission of outgoing acknowledgement so that they can be hooked onto the next outgoing data frame
- **Advantage:** higher channel bandwidth utilization
- **Complication:**
 - How long to wait for a packet to piggyback?
 - If longer than sender timeout period, then sender retransmits
⇒ **Purpose of acknowledgement is lost**
- **Solution for timing complication**
 - If a new packet arrives quickly
⇒ **Piggybacking**
 - If no new packet arrives after a receiver ACK timeout
⇒ Sending a **separate acknowledgement frame**

Sliding Window Protocols

- The next three protocols are **bidirectional protocols** that belong to a class called sliding window protocols. (max sending window size, receiving window size)
 1. One-bit sliding window protocol (1, 1)
 2. Go back N (>1, 1)
 3. Selective repeat (>1, >1)
- The three differ among themselves in terms of efficiency, complexity, and buffer requirements.
- **Each outbound frame contains an n -bit sequence number**
 - **Range: 0 - MAX_SEQ** ($\text{MAX_SEQ} = 2^n - 1$)
 - **For stop-and-wait, $n = 1$** , restricting the sequence numbers to 0 and 1 only

Sending & Receiving Windows

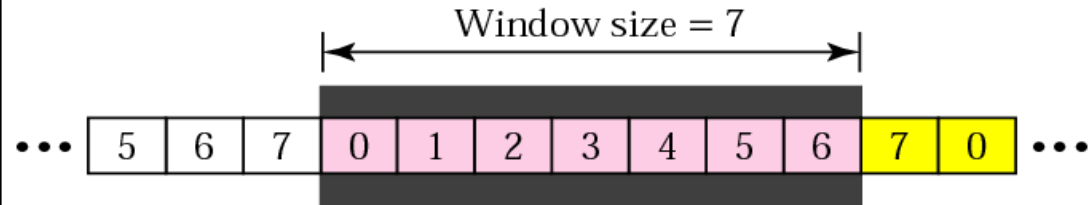
- At any instance of time
 - Sender maintains a set of sequence numbers of frames *permitted to send*
 - These frames fall within *sending window*
 - Receiver maintains a set of sequence numbers of frames *permitted to accept*
 - These frames fall within *receiving window*
- Lower limit, upper limit, and size of two windows *need not be the same* - Fixed or variable size
- **Senders Window** contains frames can be sent or have been sent but not yet acknowledged – *outstanding* frames
- **When a packet arrives from network layer**
 - Next highest sequence number assigned
 - Upper edge of window advanced by 1
- **When an acknowledgement arrives**
 - Lower edge of window advanced by 1

- If the maximum window size is n , the sender needs n buffers to hold the unacknowledged frames.
- If the window ever grows to its maximum size, the sending data link layer must forcibly shut off the network layer until another buffer becomes free.
- The receiving data link layer's window corresponds to the frames it may accept.
- Any frame falling outside the window is discarded without comment.
- When a frame whose sequence number is equal to the lower edge of the window is received, it is passed to the network layer, an acknowledgement is generated, and the window is rotated by one.

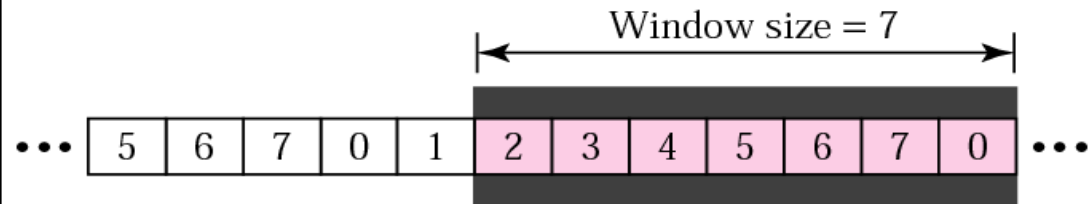
Unlike the sender's window, the receiver's window always remains at its initial size !

Sender Sliding Window

- At the sending site, to hold the outstanding frames until they are acknowledged, we use the concept of a window.
- The size of the window is at most $2^m - 1$ where m is the number of bits for the sequence number.
- Size of the window can be variable, e.g. TCP.
- The window slides to include new unsent frames when the correct ACKs are received



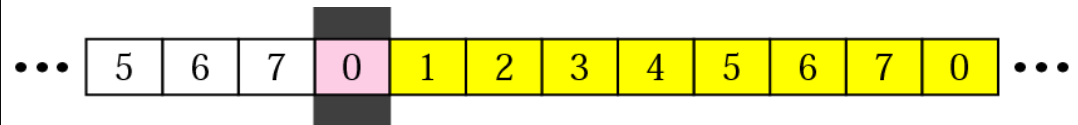
a. Before sliding



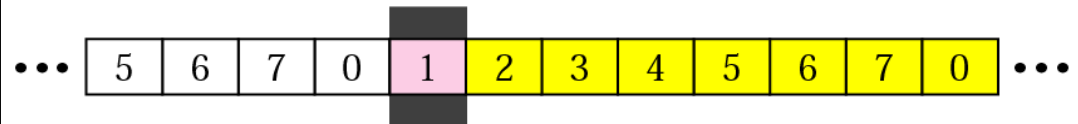
b. After sliding two frames

Receiver Sliding Window

- Size of the window at the receiving site is always 1 in this protocol.
- Receiver is always looking for a specific frame to arrive in a specific order.
- Any frame arriving out of order is discarded and needs to be resent.
- Receiver window slides as shown in fig.
 - Receiver is waiting for frame 0 in part a.



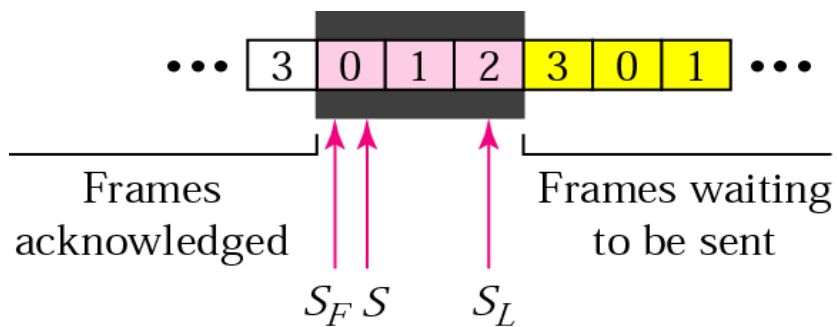
a. Before sliding



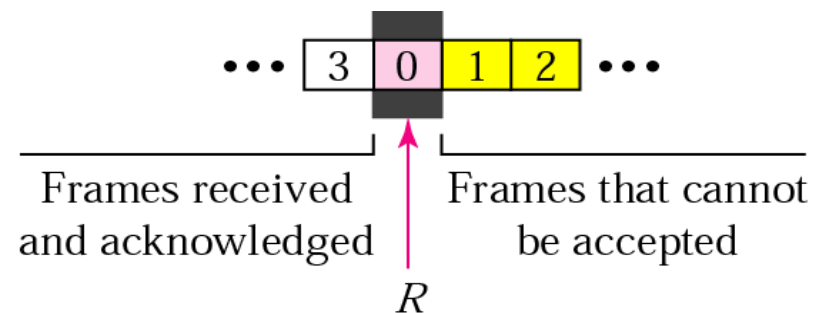
b. After sliding

Control Variables

- Sender has 3 variables: S , S_F , and S_L
- S holds the sequence number of **recently sent frame**
- S_F holds the sequence number of the **first frame**
- S_L holds the sequence number of the **last frame**
- Receiver only has the one variable, R , that holds the sequence number of the **frame it expects to receive**. If the seq. no. is the same as the value of R , the frame is accepted, otherwise rejected.

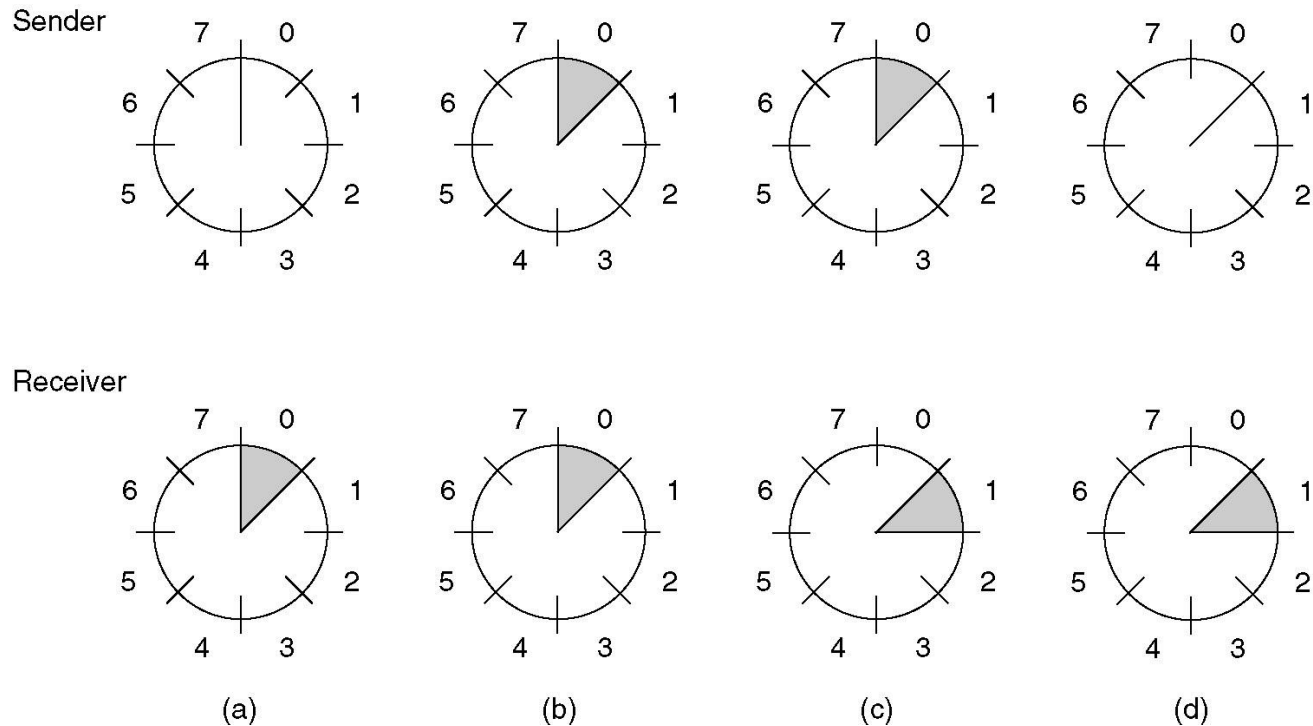


a. Sender window



b. Receiver window

1. A One-Bit Sliding Window Protocol



A sliding window of size 1, with a 3-bit sequence number.

(a) Initially.

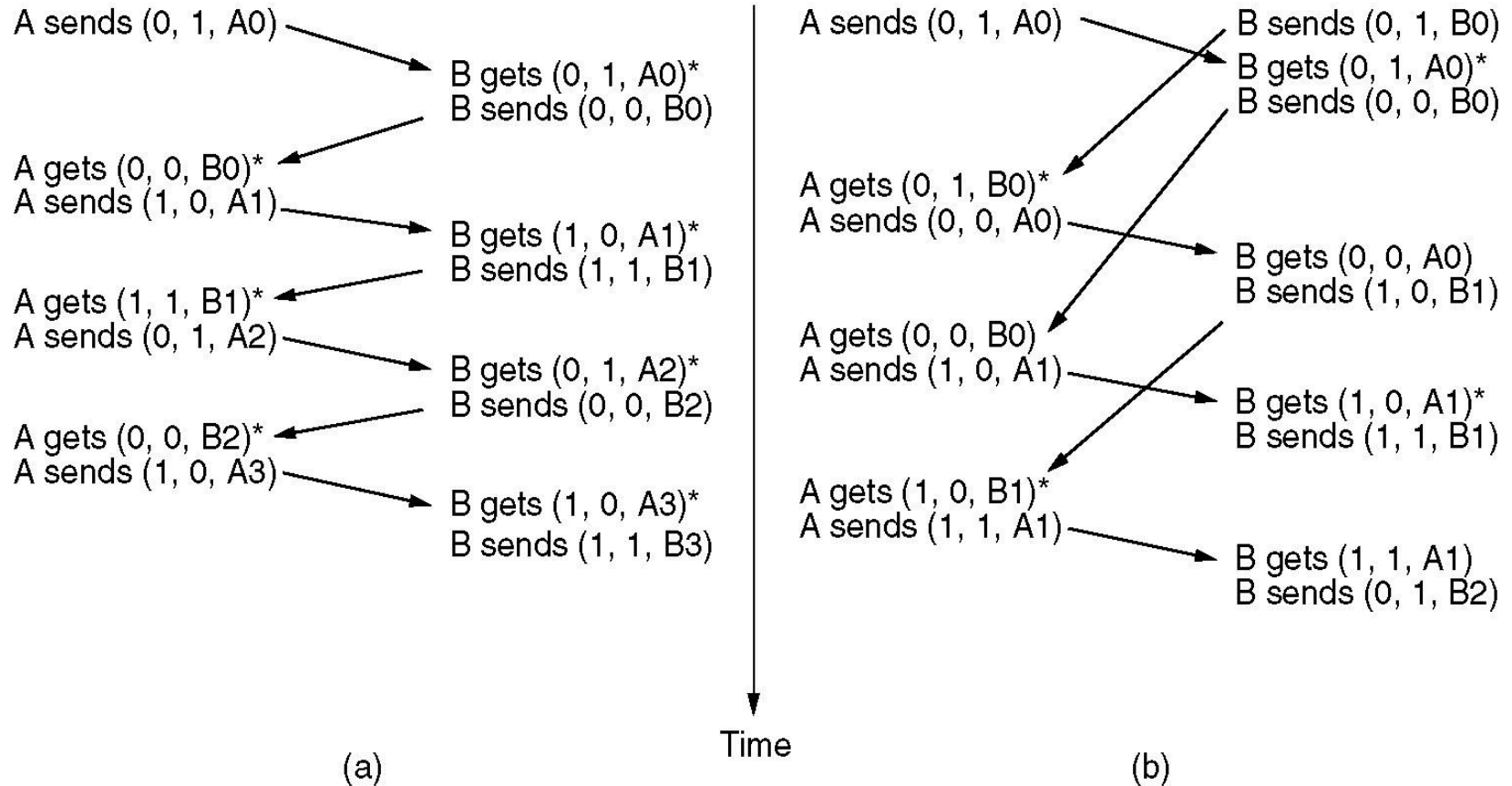
(b) After the first frame has been sent.

(c) After the first frame has been received.

(d) After the first acknowledgement has been received.

A One-Bit Sliding Window Protocol

B is piggybacking ACK for A's frame 0 to its own data B0



(a) Case 1: Normal case.

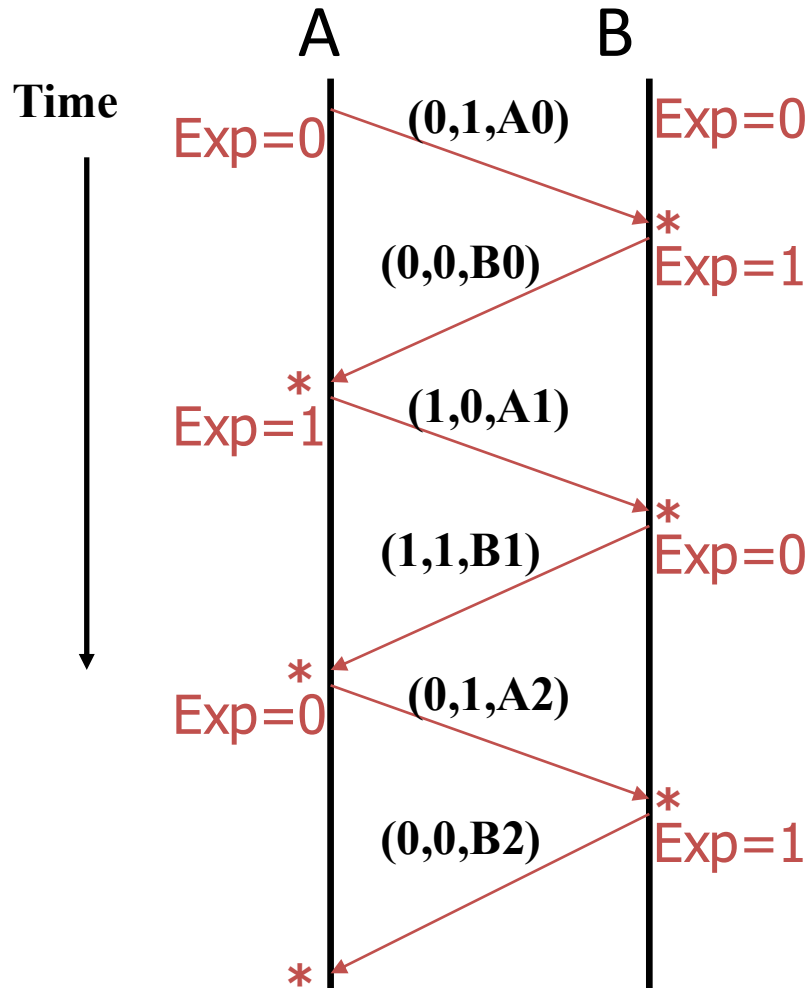
(b) Case 2: Abnormal case.

The notation is **(sequence no., acknowledgment no., packet no.)**.

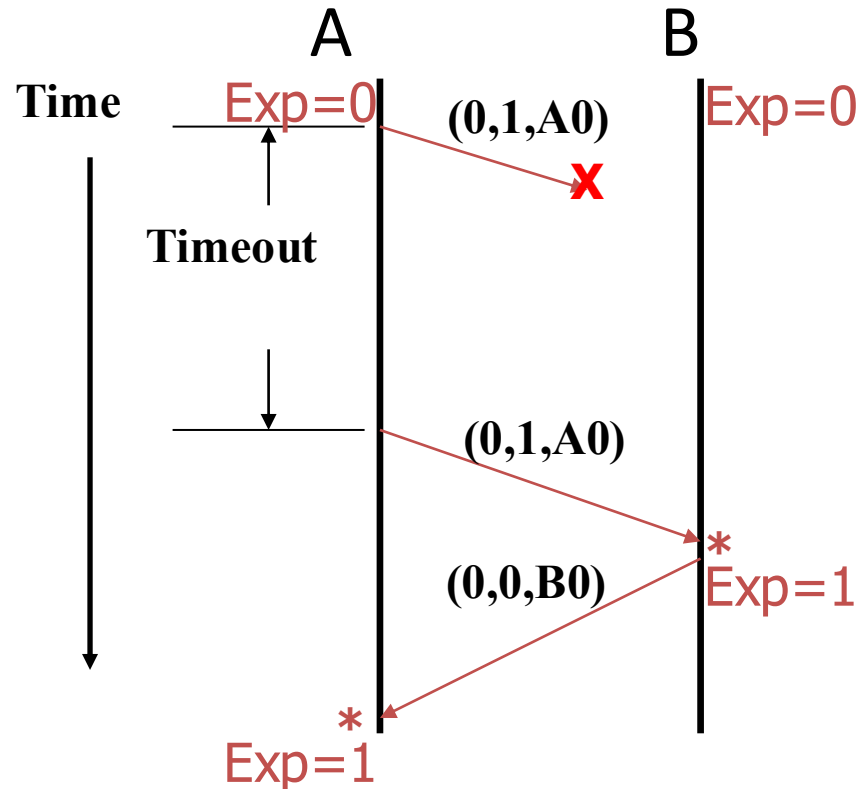
An asterisk indicates where a network layer accepts a packet.

One-Bit Sliding Window Protocol

- Case 1: no error

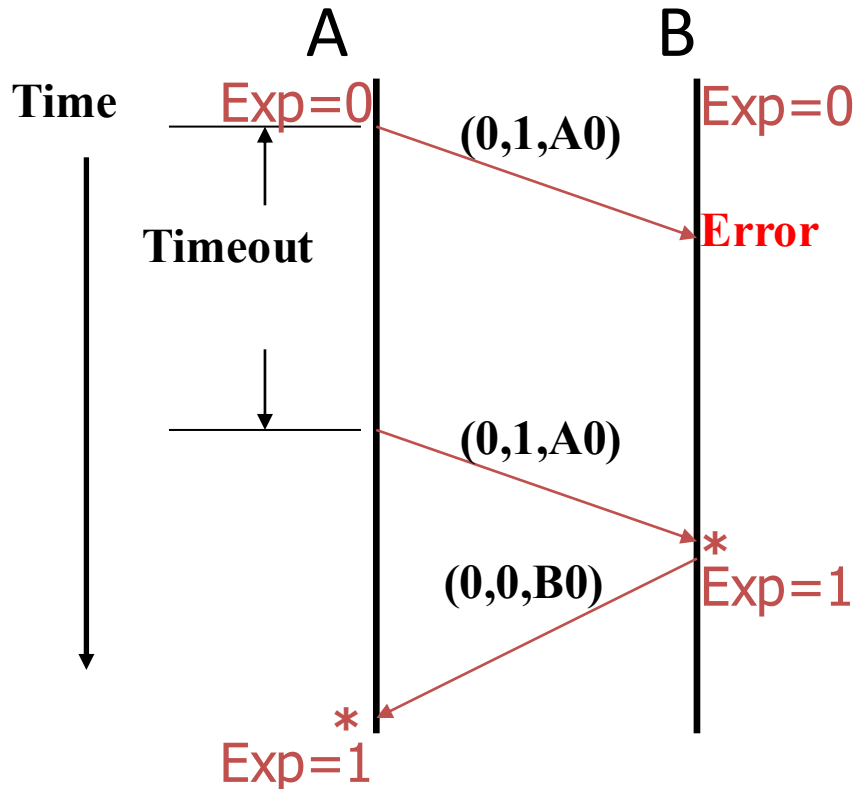


- Case 2: data lost

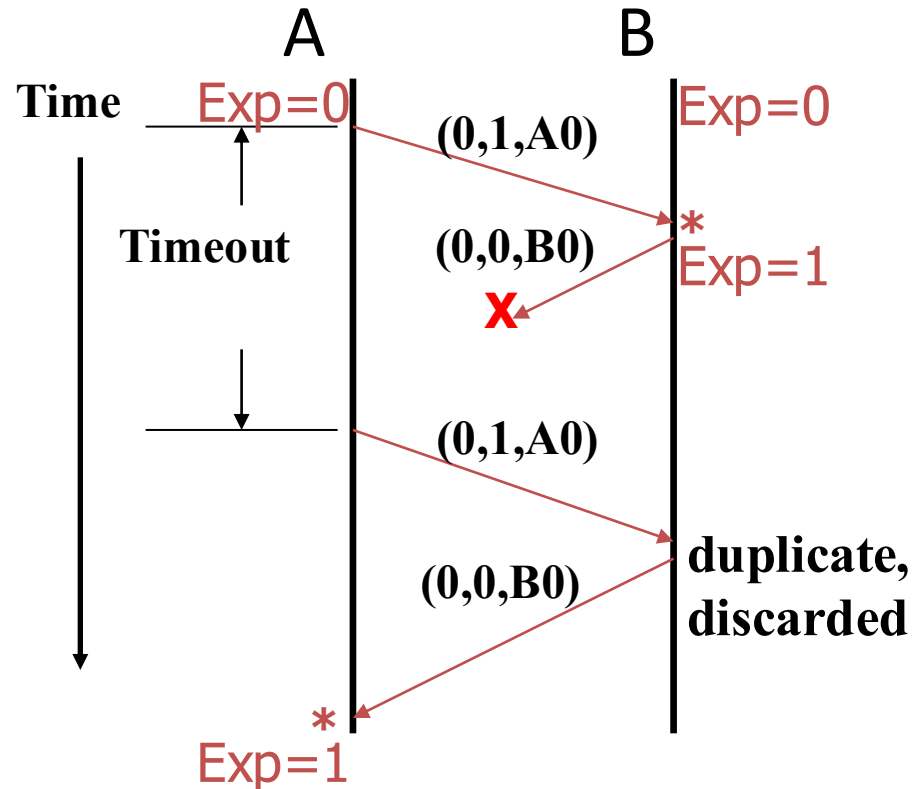


One-Bit Sliding Window Protocol

- Case 3: data error

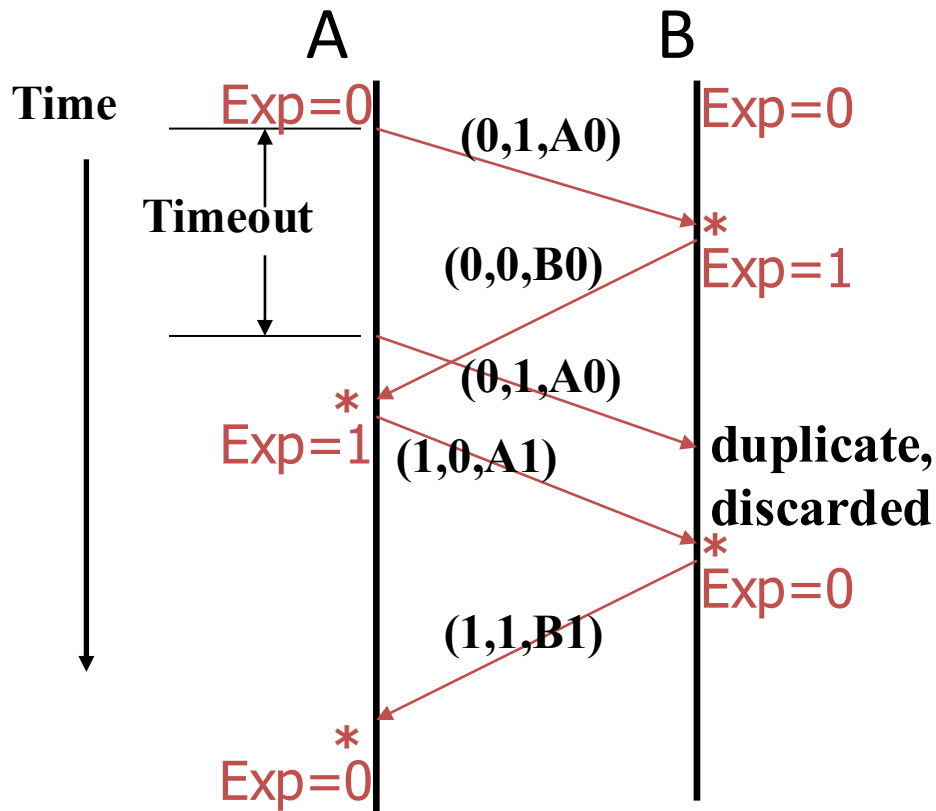


- Case 4: ack. lost

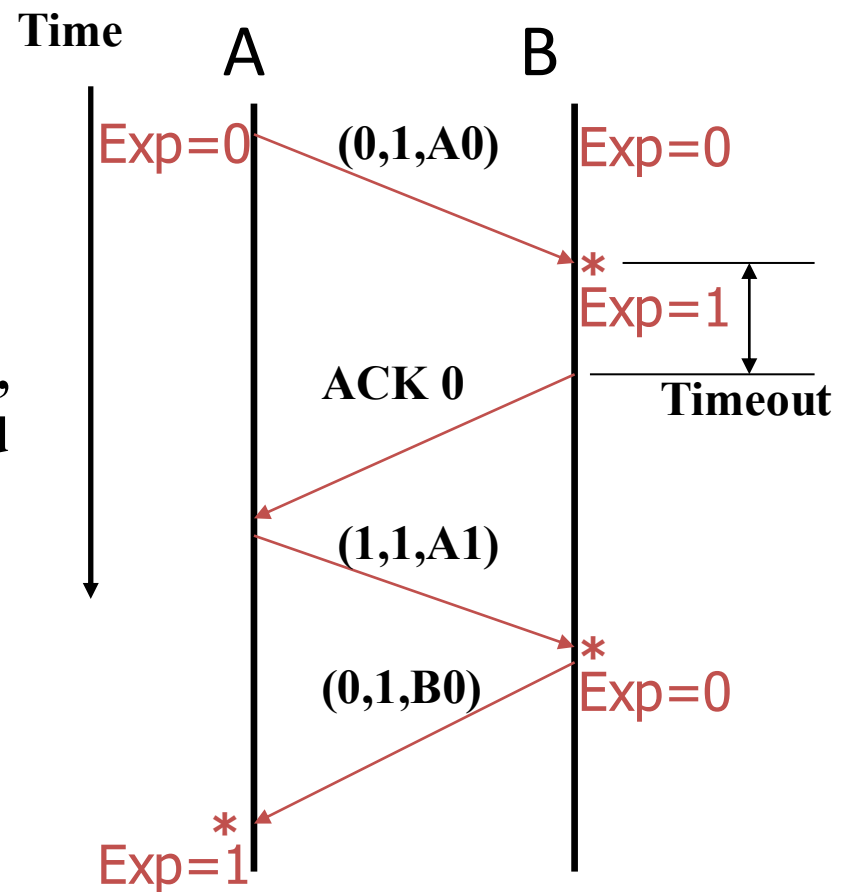


One-Bit Sliding Window Protocol

- Case 5: early timeout



- Case 6: outgoing frame timeout



Performance of Stop-and-Wait Protocol

- Previous protocols can be preferred when transmission time is negligible.
- However, they **cannot be preferred when transmission time is long.**
- **Example - satellite communication**

- channel capacity/bandwidth: 50 kbps, frame size: 1kb
- round-trip propagation delay: 500 msec : $[T_X \rightarrow R_X \rightarrow T_X = 500 \text{ ms}]$

How much time does the sender spend actually transmitting, and how much time does it spend simply waiting?

- Time: $t=0$ start to send 1st bit in frame
- $t=20 \text{ msec}$ frame sent completely :

$$T_{\text{transmission}}(\text{sec}) = \frac{\text{frame size (kb)}}{\text{channel capacity (kbps)}} = \frac{1000}{50000} = 20\text{ms}$$

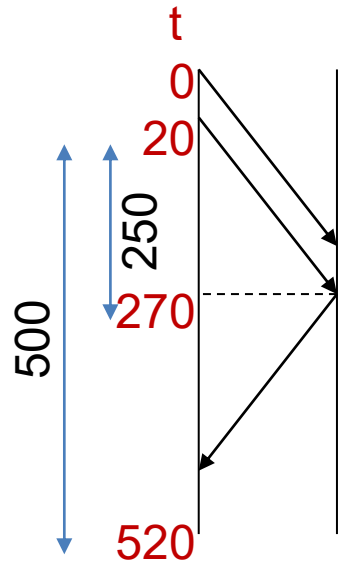
$t=270 \text{ msec}$ frame arrives

$t=520 \text{ msec}$ best case of ack. Received

- Sender blocked $500/520 = 96\%$ of time
 - Bandwidth utilization $20/520 = 4\%$

Conclusion:

Long transit time + high bandwidth + short frame length $\Rightarrow$ **disaster**



Performance of 1-bit sliding window Protocol

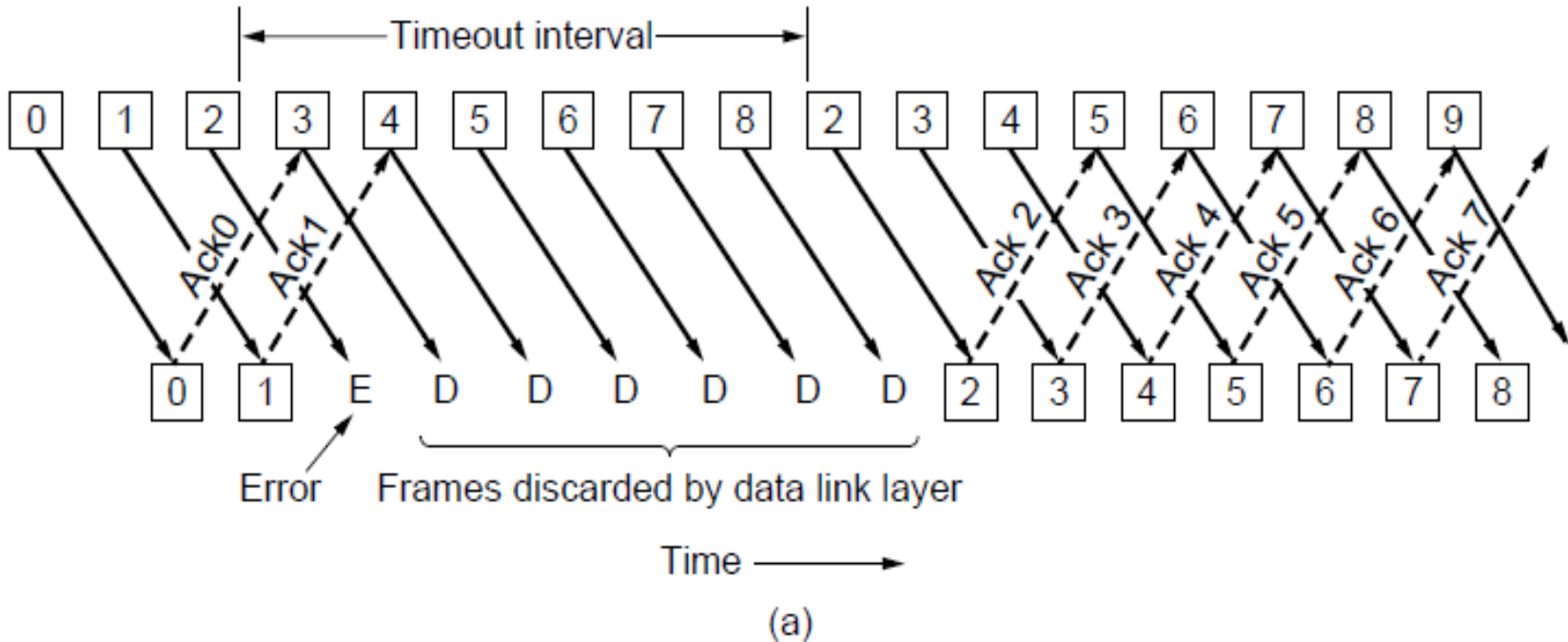
- In stop-and-wait, at any point in time, there is only one frame that is sent and waiting to be acknowledged.
- This is not a good use of transmission medium.
- To improve efficiency, multiple frames should be in transition while waiting for ACK.
- Solution: **PIPELINING**
 - Allowing w frames sent before blocking
- Problem: errors
- Solutions
 - Go back n protocol (GNP)
 - Selective repeat protocol (SRP)

Acknowledge n means frames $n, n-1, n-2, \dots$ are acknowledged (i.e., received correctly)

2. Go Back n Protocol

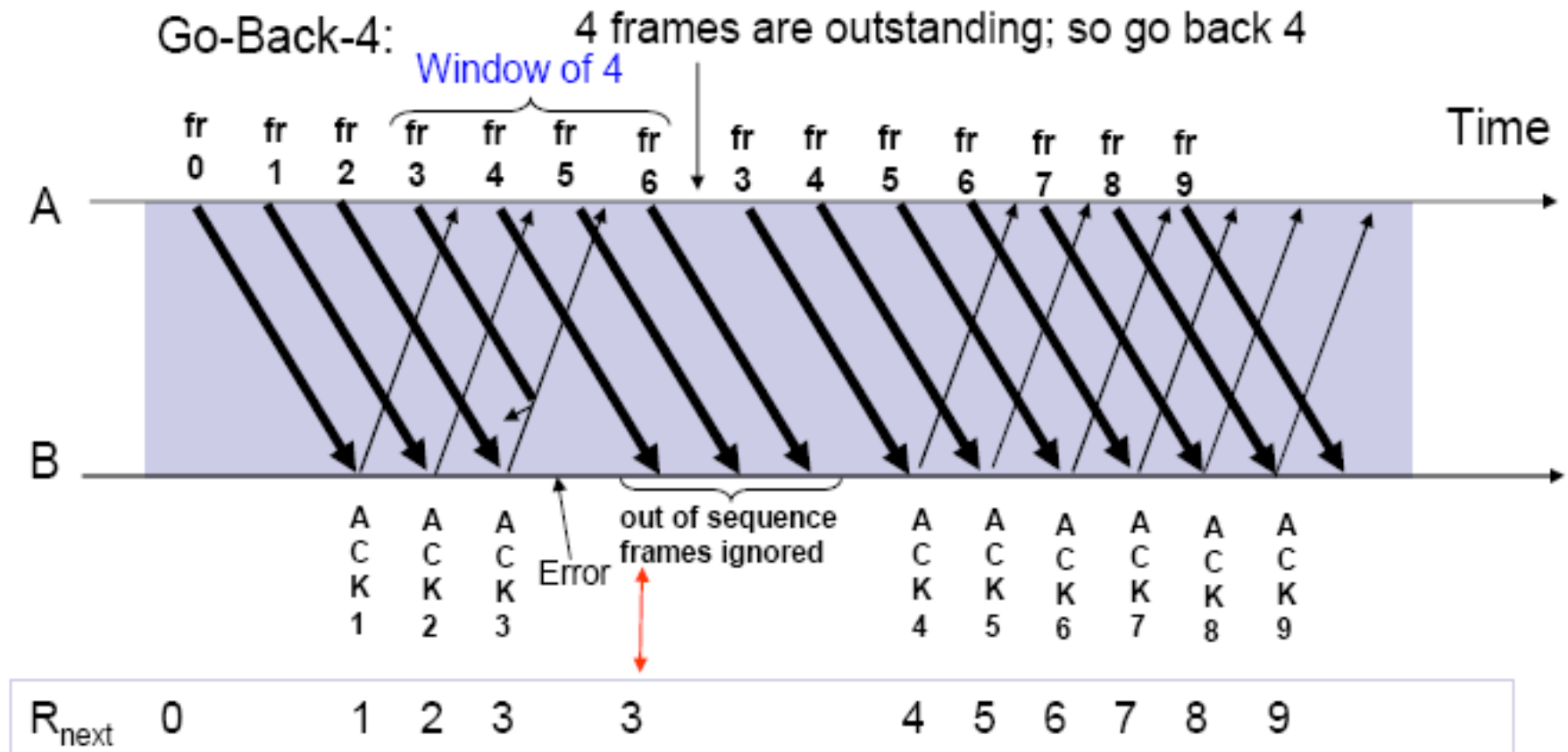
- Improves efficiency of Stop and Wait by not waiting
- Keep Channel busy by continuing to send frames
- Allow a window of up to w_s outstanding frames
- Use **m-bit sequence numbering**
- Receiver **discards all subsequent frames following an error one**, and **send no acknowledgement** for those discarded
- **Receiving window size = 1** (i.e., frames must be accepted in the order they were sent)
- Sending window might get full
 - If so, re-transmitting unacknowledged frames
- **Wasting a lot of bandwidth if error rate is high**

Go Back n Protocol



- Frames 0 and 1 are correctly received and acknowledged. Frame 2, however, is damaged or lost.
- The sender, unaware of this problem, continues to send frames until the timer for frame 2 expires.
- Then it backs up to frame 2 and starts all over with it, sending 2, 3, 4, etc. all over again.

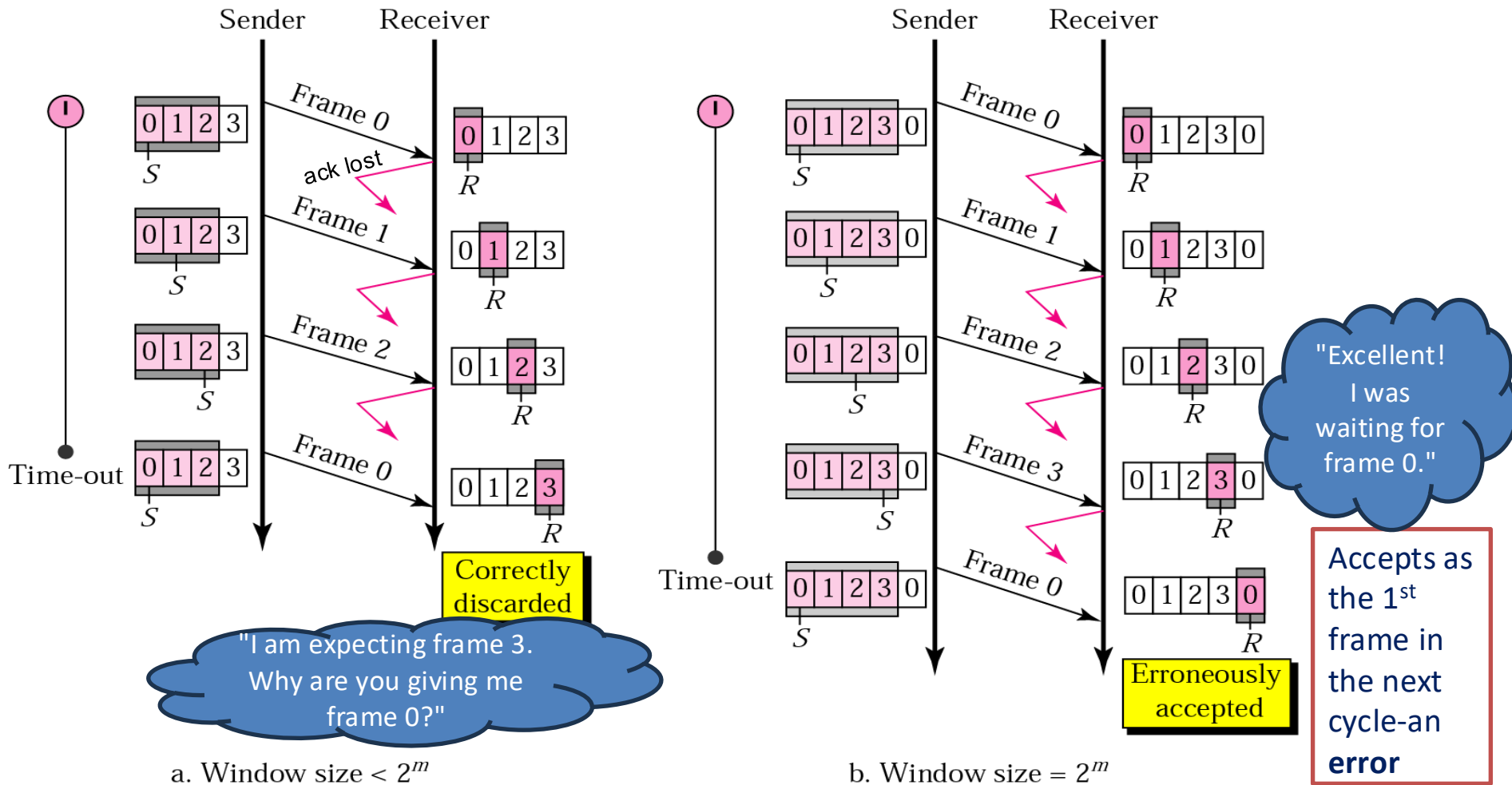
Go-Back-N ARQ with window=4



- Frame transmission are *pipelined* to keep the channel busy
- Frame with errors and subsequent out-of-sequence frames are ignored
- Transmitter is forced to go back when **window of 4** is exhausted

Go-Back-N ARQ, sender window size

- Size of the sender window must be less than 2^m . Size of the receiver is always 1. If $m = 2$, window size = $2^m - 1 = 3$.
- Fig. compares a window size of 3 and 4.



Consider the Go back N protocol with a sender's window size of ' n '. Suppose that at time ' t ', the next inorder packet the receiver is expecting has a sequence number of ' K '. Assume that the medium does not reorder messages.

What are the possible sets of sequence numbers inside the sender's window at time ' t '. Assume the sender has already received the ACKs.

Answer:

$[K, K+n-1]$

- Station A needs to send a message consisting of 9 packets to station B using a sliding window (window size 3) and go back n error control strategy. All packets are ready and immediately available for transmission.
- If every 5th packet that A transmits gets lost (but no ACKs from B ever get lost), then what is the number of packets that A will transmit for sending the message to B?

- 1,2,3,4,5,6,7,8,9 have to be sent

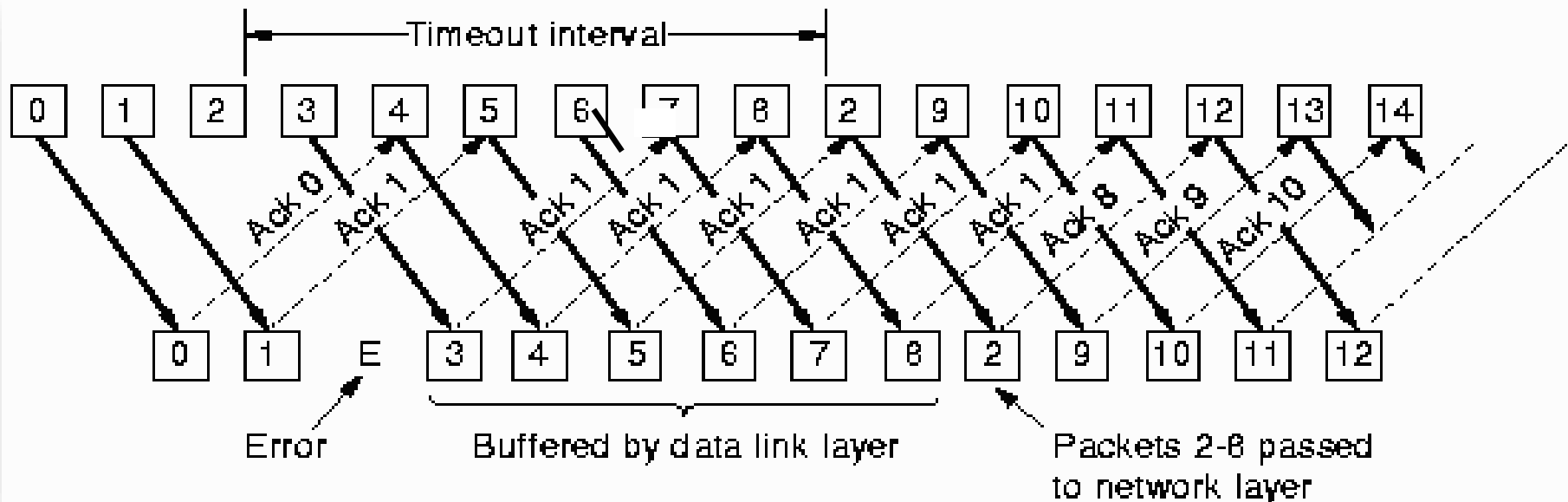
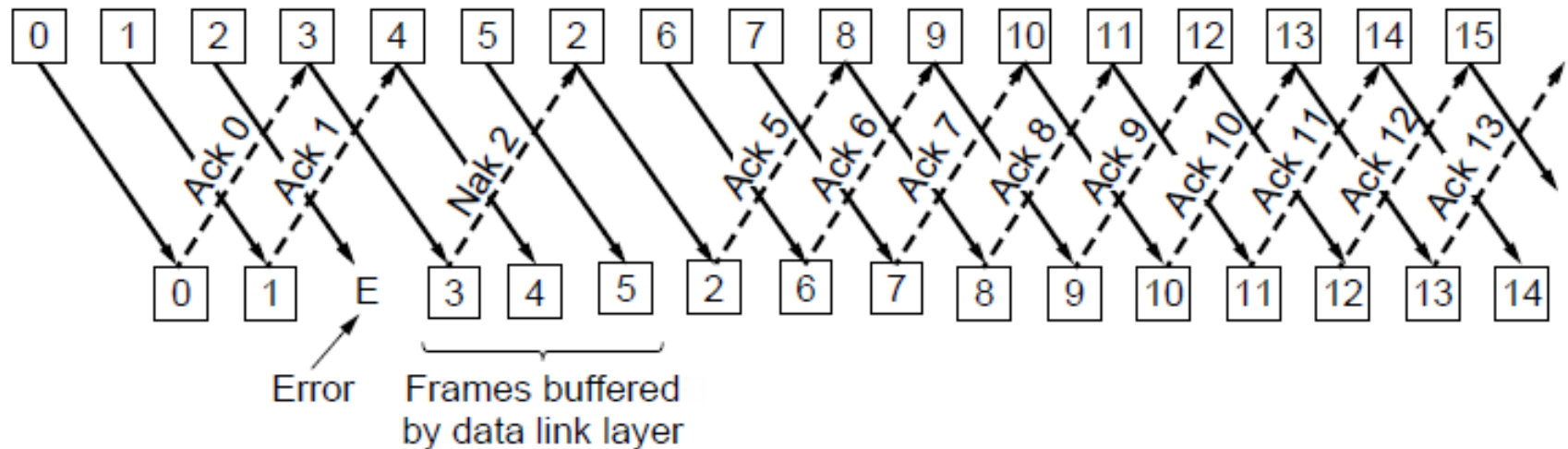
Sender Window			Receiver Window	Total packets sent
3	2	1	1	3
4	3	2	2	4
5	4	3	3	5
6	5	4	4	6
7	6	5		7
7	6	5	5	10
8	7	6	6	11
9	8	7		12
9	8	7	7	15
	9	8	8	
		9		
		9	9	16

- In Go back 4, if every 6th packet that is being transmitted is lost and if total number of packets to be sent is 10, then how many transmissions will be required?
- Solution: 17

3. Select Repeat Protocol

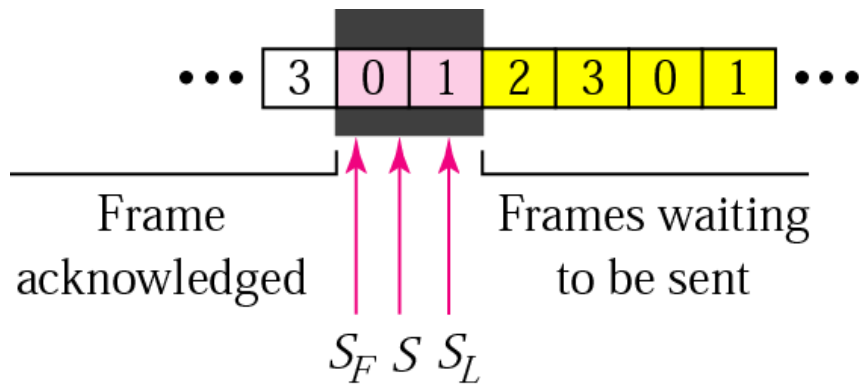
- Receiver stores correct frames following the bad one
- Sender retransmits the bad one after noticing
- Receiver passes data to network layer and acknowledge with the highest number
- Receiving window > 1 (i.e., any frame within the window may be accepted and buffered until all the preceding one passed to the network layer.)
 - Might need large memory
- ACK for frame n implicitly acknowledges all frames $\leq n$ – *COLLECTIVE ACK*
- When error is *suspected* by receiver, receiver request retransmission of a frame
 - Arrival of a damaged frame
 - Arrival of a frame other than the expected
- NAKs stimulate retransmission before the corresponding timer expires and thus improve performance.

Selective Repeat with NAK

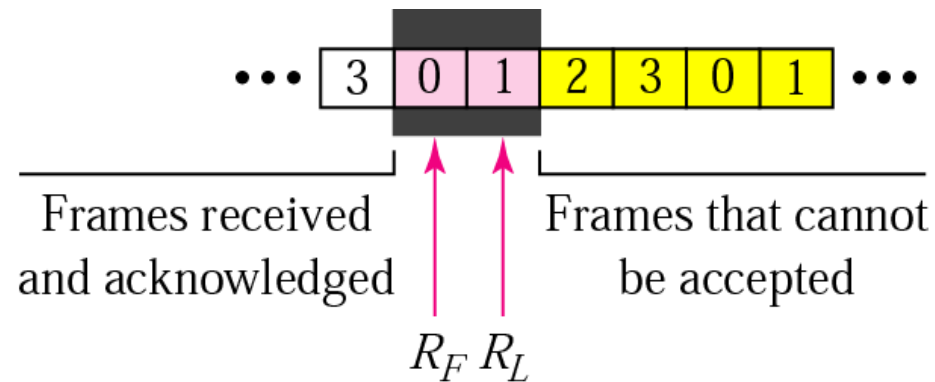


Selective Repeat ARQ, sender and receiver windows.

- **Go-Back-N ARQ** simplifies the process at the receiver site. Receiver only keeps **track of only one variable**, and there is **no need to buffer** out-of-order frames, they are simply discarded.
- However, Go-Back-N ARQ protocol is **inefficient for noisy link**. It is **bandwidth-inefficient** and slows down the transmission.
- In **Selective Repeat ARQ**, only the damaged frame is resent. **More bandwidth-efficient** but more complex processing at receiver.
- It defines a negative ACK (NAK) to report the sequence number of a damaged frame before the timer expires.

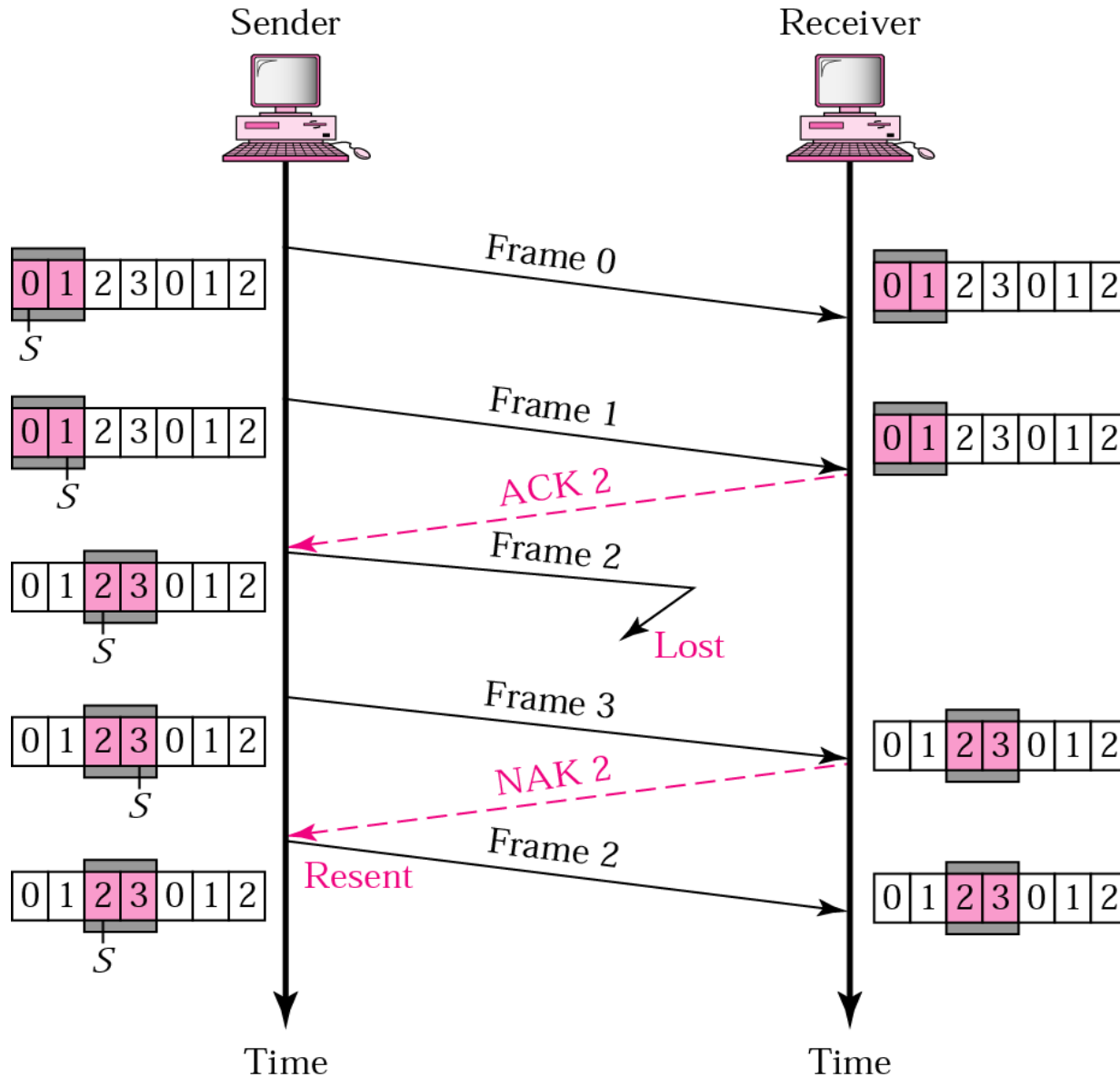


a. Sender window



b. Receiver window

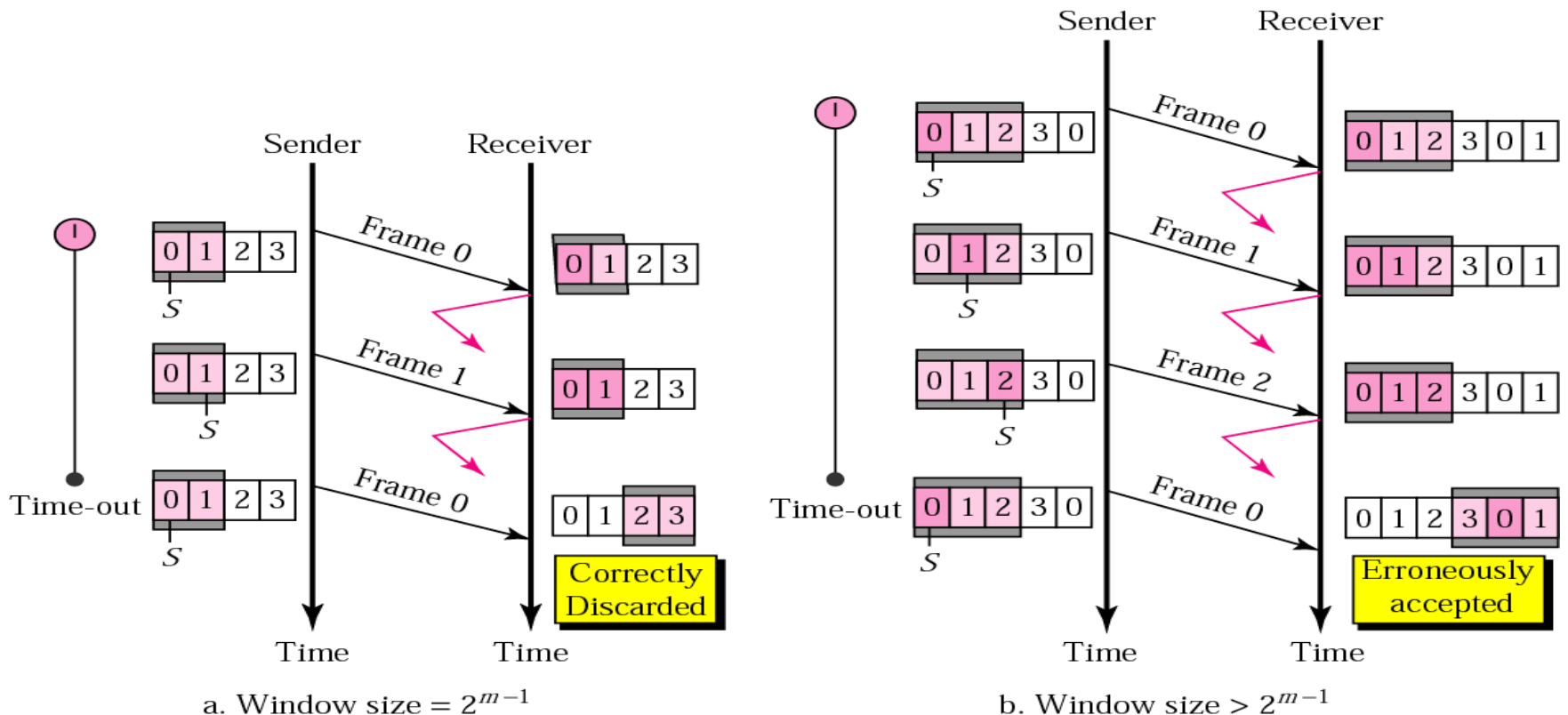
Selective Repeat ARQ, lost frame



- Frames 0 and 1 are accepted when received because they are in the range specified by the receiver window. Same for frame 3.
- Receiver sends a NAK2 to show that frame 2 has not been received and then sender resends only frame 2 and it is accepted as it is in the range of the window.

Selective Repeat ARQ, sender window size

- Size of the sender and receiver windows must be at most one-half of 2^m .
- If $m = 2$ (i.e. sequence number has 2 bits), window size should be $2^m/2 = 2^{m-1}$.
Fig compares a window size of 2 with a window size of 3. Window size is 3 and all ACKs are lost, sender sends duplicate of frame 0, window of the receiver expect to receive frame 0 (part of the window), so accepts frame 0, as the 1st frame of the next cycle – an **error**.



In SR protocol, suppose frames through 0 to 4 have been transmitted. Now, imagine that 0 times out, 5 (a new frame) is transmitted, 1 times out, 2 times out and 6 (another new frame) is transmitted.

At this point, what will be the outstanding packets in sender's window?

Answer –

4,3,2,1,0

0 times out

0,4,3,2,1

5 transmitted

5,0,4,3,2,1

1 times out

1,5,0,4,3,2

2 times out

2,1,5,0,4,3

6 transmitted

6,2,1,5,0,4,3

**NUMERICALS ON GO BACK N &
SELECTIVE REPEAT PROTOCOLS**

Efficiency of sliding protocols

- Both Go Back N & Selective Repeat protocols have the same efficiency:

$$\eta = \frac{\text{useful transmission time}}{\text{total cycle time}}$$

$$\eta = \min \left(1, \frac{N}{1 + 2a} \right)$$

Where N = window size

$$a = \frac{T_p}{T_t}$$

20 Kbps satellite link has a propagation delay of 400 ms. The transmitter employs the “go back n ARQ” scheme with n set to 10.

Assuming that each frame is 100 bytes long, what is the maximum data rate possible?

$$\mathbf{BW} = 20\text{Kbps} = 20 \times 10^3 \text{ bps}$$

$$\mathbf{Frame\ size} = 100 \text{ bytes} = 100 \times 8 = 800 \text{ bits}$$

$$\mathbf{Propagation\ Delay} (T_p) = 400 \text{ msec}$$

$$\mathbf{N=10}$$

$$T_t = \frac{\text{Frame size}}{BW} = \frac{800}{20000} = 40 \text{ msec}$$

$$\begin{aligned}\eta &= \min\left(1, \frac{N}{1+2\left(T_p/T_t\right)}\right) \\ &= \min\left(1, \frac{10}{1+2\left(400/40\right)}\right) \\ &= \min\left(1, \frac{10}{1+20}\right) \\ &= \min\left(1, \frac{10}{1+20}\right) = \min(1, 0.476) = 0.476\end{aligned}$$

$$\mathbf{Throughput} = \eta \times BW = 0.476 \times 20 \times 10^3 \text{ bps} = 9520 \text{ bits} = 9.52 \text{ Kbps}$$

Consider a 128×10^3 bits/sec satellited communication link with one way propagation delay of 150 msec. Sliding protocol is used on this link to send data with a frame size of 1 KB. Neglect the transmission time of acknowledgement.

The minimum number of bits required for the sequence number field to achieve 100% utilization is _____ .

$$\text{Frame size} = 1\text{KB} = 2^{10} \times 8 = 1024 \times 8 = 8192 \text{ bits}$$

Propagation Delay (T_p)-

$$T_p = 150 \text{ msec}$$

$$\text{RTT} = 150 \text{ msec} \times 2 = 300 \text{ msec}$$

$$\begin{aligned} \text{Bits during 1 RTT} &= \text{RTT} \times \text{BW} \\ &= 300 \text{ msec} \times 128 \text{ Kbps} \\ &= 38400 \text{ bits} \end{aligned}$$

$$\begin{aligned} \text{Total count of frames} &= \frac{38400}{8192} \\ &= 4.6875 \\ &\cong 5 \end{aligned}$$

$$\text{Window Size} \leq 2^m - 1$$

$$5 \leq 2^m - 1$$

$$6 \leq 2^m$$

$$\text{For } m=3, 2^3 = 8 \geq 6$$

Go Back N

Sequence Number used should be 3 bit long in Go Back N protocol.

Selective Repeat

$$\begin{aligned}\text{Total count of frames} &= \frac{38400}{8192} \\ &= 4.6875 \\ &\cong 5\end{aligned}$$

$$\text{Window Size} \leq 2^{m-1}$$

$$5 \leq 2^{m-1}$$

$$\text{For } m=4, 2^{4-1} = 2^3 = 8 \geq 5$$

Sequence Number used should be 4 bit long in Selective Repeat protocol.

A 3000 km long trunk operates at 1.536 Mbps and is used to transmit 64 byte frames and uses sliding window protocol. If the propagation speed is 6 $\mu\text{sec} / \text{km}$, how many bits should the sequence number field be?

Frame size = 64B = $64 \times 8 = 512$ bits

Go Back N

Propagation Delay (T_p)-

For 1 km, $T_p = 6 \mu\text{sec}$

For 3000 km, $T_p = 3000 \times 6 \mu\text{sec} = 18000 \mu\text{sec}$

RTT = $18000 \mu\text{sec} \times 2 = 36000 \mu\text{sec}$

Sequence Number used
should be 7 bit long in Go
Back N protocol.

Bits during 1 RTT = $\text{RTT} \times \text{BW}$
= $36000 \mu\text{sec} \times 1.536 \text{ Mbps}$
= 553296 bits

Total count of frames = $\frac{553296}{512}$
= 108

Window Size $\leq 2^m - 1$

$$108 \leq 2^m - 1$$

$$109 \leq 2^m$$

For $m=7$, $2^7 = 128 \geq 109$

Selective Repeat

$$\begin{aligned}\text{Total count of frames} &= \frac{553296}{512} \\ &= 108\end{aligned}$$

$$\text{Window Size} \leq 2^{m-1}$$

$$108 \leq 2^{m-1}$$

$$\text{For } m=8, 2^{8-1} = 2^7 = 128 \geq 108$$

Sequence Number used should be 8 bit long in Selective Repeat protocol.

Compute approximate optimal window size when packet size is 53 bytes, RTT is 60 msec and bottleneck bandwidth is 155 Mbps.

$$\text{Frame size} = 53\text{B} = 53 \times 8 = 424 \text{ bits}$$

$$\text{RTT} = 60 \text{ msec}$$

$$\text{BW} = 155 \text{ Mbps}$$

$$\begin{aligned} \text{Bits during 1 RTT} &= \text{RTT} \times \text{BW} \\ &= 60 \text{ msec} \times 155 \text{ Mbps} \\ &= 9300000 \text{ bits} \end{aligned}$$

$$\begin{aligned} \text{Total count of frames} &= \frac{9300000}{424} \\ &= 21933.96 \\ &\cong 21934 \end{aligned}$$

Multiple Access Protocols

Introduction

- When a sender and receiver have a dedicated link to transmit data packets, the data link control is enough to handle the channel.
- Suppose there is no dedicated path to communicate or transfer the data between two devices.
- For example, suppose that there is a classroom full of students.
 - When a teacher asks a question, all the students (small channels) in the class start answering the question at the same time (transferring the data simultaneously).
 - All the students respond at the same time due to which data is overlap or data lost.
 - Therefore, it is the responsibility of a teacher (multiple access protocol) to manage the students and make them one answer.

Multiple Access Protocols

```
graph TD; A[Multiple Access Protocols] --> B[Random Access Protocols]; A --> C[Controlled Access Protocols]; A --> D[Channelization Protocols]; B --> B1[ALOHA]; B --> B2[CSMA]; B --> B3[CSMA/CD]; B --> B4[CSMA/CA]; C --> C1[Reservation]; C --> C2[Polling]; C --> C3[Token Passing]; D --> D1[FDMA]; D --> D2[TDMA]; D --> D3[CDMA];
```

Random Access Protocols

ALOHA

CSMA

CSMA/CD

CSMA/CA

Controlled Access Protocols

Reservation

Polling

Token Passing

Channelization Protocols

FDMA

TDMA

CDMA

Random Access Protocols

- Random access protocols assign uniform priority to all connected nodes.
- Any node can send data if the transmission channel is idle.
- No fixed time or fixed sequence is given for data transmission.
- The four random access protocols are–
 - ALOHA
 - Carrier sense multiple access (CMTA)
 - Carrier sense multiple access with collision detection (CMTA/CD)
 - Carrier sense multiple access with collision avoidance (CMTA/CA)

Controlled Access Protocols

- Controlled access protocols allow only one node to send data at a given time.
- Before initiating transmission, a node seeks information from other nodes to determine which station has the right to send.
- This avoids collision of messages on the shared channel.
- The station can be assigned the right to send by the following three methods–
 - Reservation
 - Polling
 - Token Passing

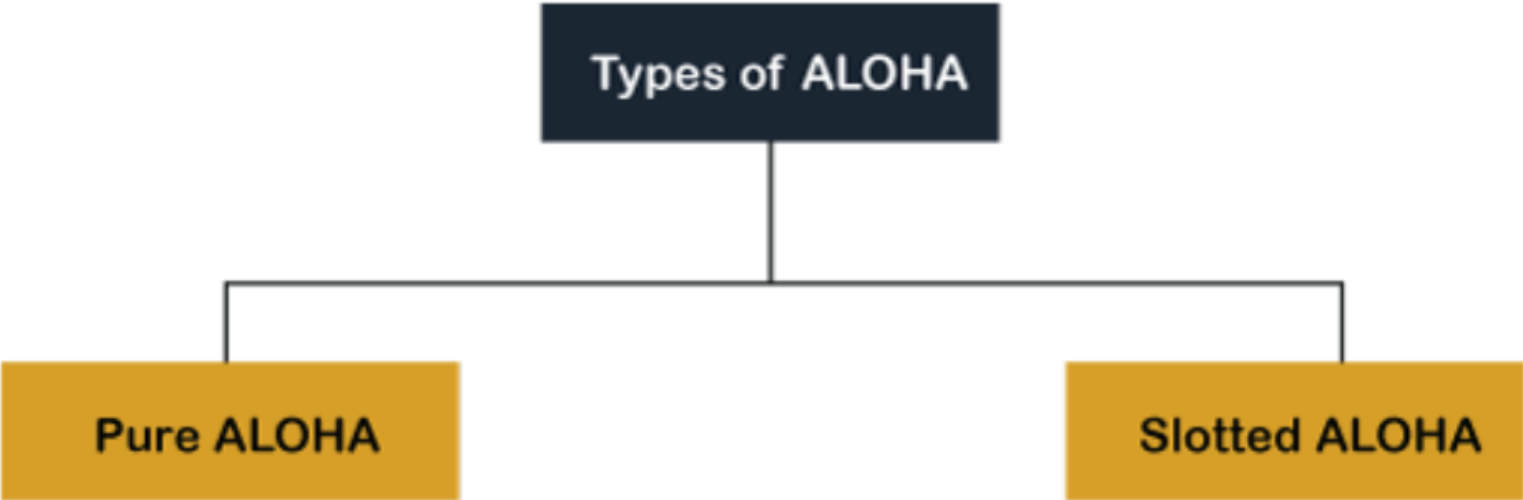
Channelization Protocols

- Channelization are a set of methods by which the available bandwidth is divided among the different nodes for simultaneous data transfer.
- The three channelization methods are–
 - Frequency division multiple access (FDMA)
 - Time division multiple access (TDMA)
 - Code division multiple access (CDMA)

ALOHA Random Access Protocol

- It is **designed for wireless LAN** (Local Area Network) but can also be used in a shared medium to transmit data.
- Using this method, any station can transmit data across a network simultaneously when a data frameset is available for transmission.
- **Aloha Rules**
 - Any station can transmit data to a channel at any time.
 - It does not require any carrier sensing.
 - Data frames may be lost during the transmission of data through multiple stations.
 - **Acknowledgment of the frames exists in Aloha. Hence, there is collision detection.**
 - It requires retransmission of data after some random amount of time.

Types of ALOHA



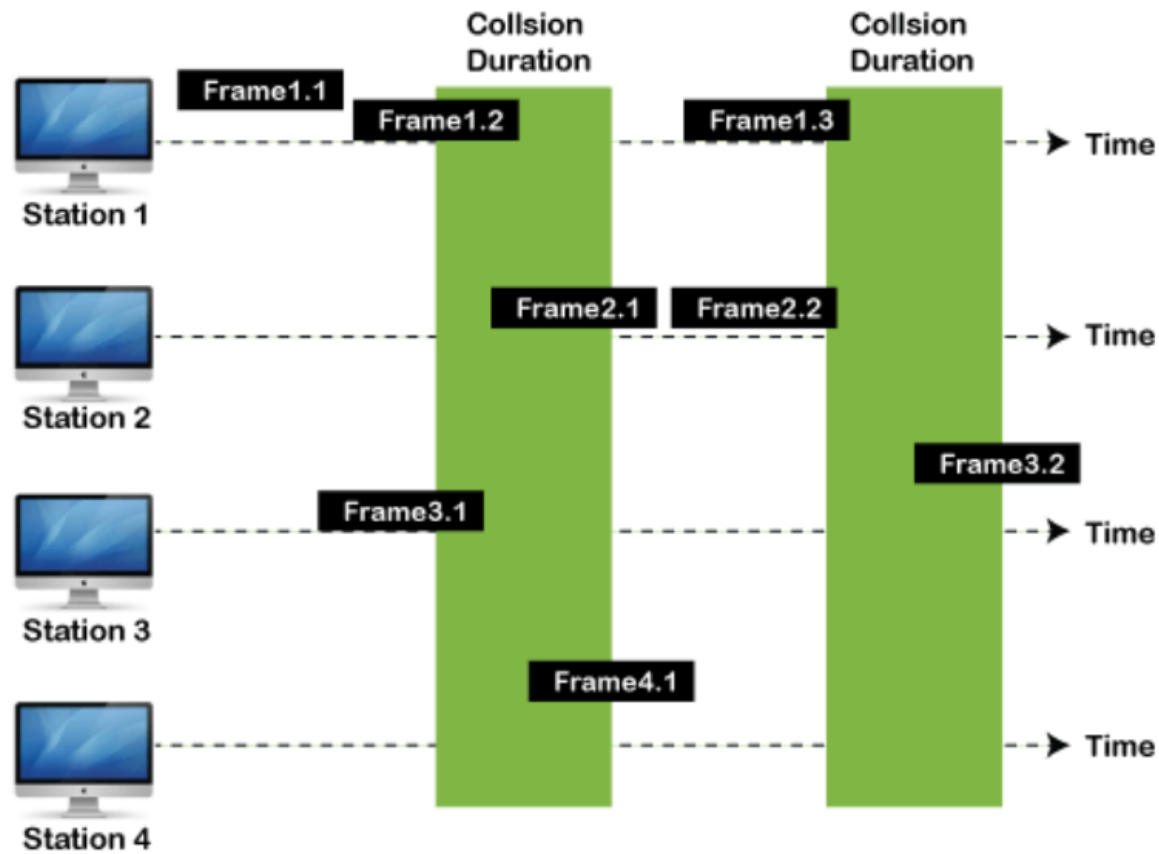
```
graph TD; A[Types of ALOHA] --> B[Pure ALOHA]; A --> C[Slotted ALOHA]
```

Pure ALOHA

Slotted ALOHA

Pure ALOHA

- Whenever data is available for sending over a channel at stations, we use Pure Aloha.
- In pure Aloha, when each station transmits data to a channel without checking whether the channel is idle or not, the chances of collision may occur, and the data frame can be lost.
- When any station transmits the data frame to a channel, the pure Aloha waits for the receiver's acknowledgment.
- If it does not acknowledge the sender end within the specified time, the station waits for a random amount of time, called the backoff time (T_b).
- And the station may assume the frame has been lost or destroyed.
- Therefore, it retransmits the frame until all the data are successfully transmitted to the receiver.



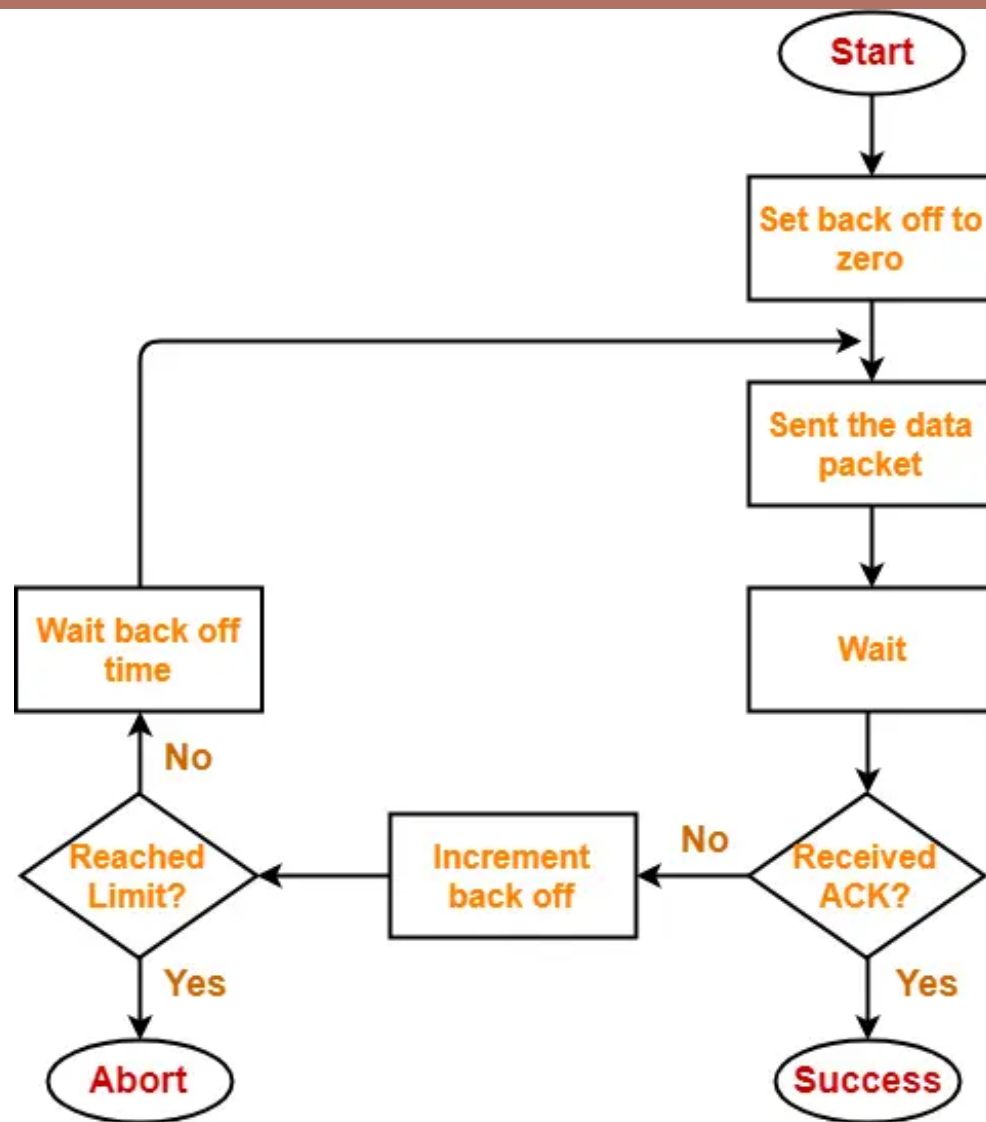
Frames in Pure ALOHA

Case-01:

- Transmitting station **receives an acknowledgement** from the receiving station.
- In this case, transmitting station assumes that the **transmission is successful.**

Case-02:

- Transmitting station **does not receive any acknowledgement** within specified time from the receiving station.
- In this case, transmitting station assumes that the **transmission is unsuccessful.**
- Transmitting station uses a **Back Off Strategy** and waits for some random amount of time.
- After back off time, it transmits the data packet again.
- It **keeps trying until the back off limit** is reached **after which it aborts the transmission.**



Flowchart for Pure Aloha

Efficiency

- **Efficiency of Pure Aloha,**

$$\eta = G \times e^{-2G}$$

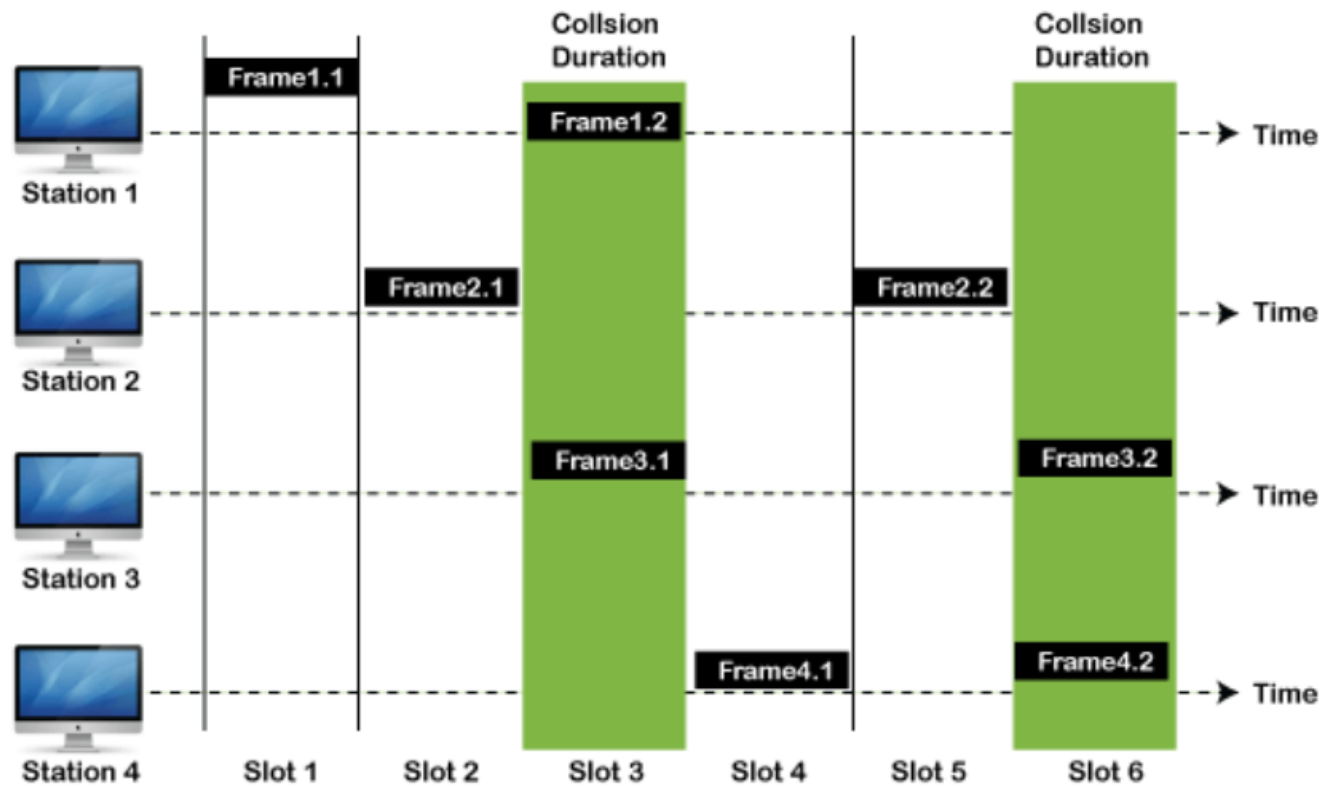
where G = offered load - how many transmission attempts are being made relative to the channel's frame capacity.

Maximum Efficiency:

- $G = 0.5$; to balance between $G < 0.5$ (too many empty slots) and $G > 0.5$ (too many collisions)
- $\eta = 0.5 \times e^{-2(0.5)} = 0.5 \times e^{-1} = \frac{0.5}{2.718} = 0.184$
- It means that **at its best**, Pure ALOHA successfully uses only about **18.4% of the channel capacity** for useful data transmission.

Slotted ALOHA

- The slotted Aloha is designed to overcome the pure Aloha's efficiency because pure Aloha has a very high possibility of frame hitting.
- In slotted Aloha, the shared channel is divided into a fixed time interval called slots.
- So that, if a station wants to send a frame to a shared channel, the frame can only be sent at the beginning of the slot, and only one frame is allowed to be sent to each slot.
- And if the stations are unable to send data to the beginning of the slot, the station will have to wait until the beginning of the slot for the next time.
- However, the possibility of a collision remains when trying to send a frame at the beginning of two or more station time slot.



Frames in Slotted ALOHA

Efficiency

- **Efficiency of Pure Aloha,**

$$\eta = G \times e^{-G}$$

where G = offered load - how many transmission attempts are being made relative to the channel's frame capacity.

Maximum Efficiency:

- $G = 1$; to balance between $G < 1$ (too many empty slots) and $G > 1$ (too many collisions)
- $\eta = 1 \times e^{-1} = \frac{1}{2.718} = 0.368$
- It means that **at its best**, Slotted ALOHA successfully uses only about **36.8% of the channel capacity** for useful data transmission.

Problem-

A group of N stations share 100 Kbps slotted ALOHA channel. Each station output a 500 bits frame on an average of 5000 ms even if previous one has not been sent.

What is the required value of N ?

Throughput Of One Station-

- Throughput of each station = Number of bits sent per second
= 500 bits / 5000 ms
= 500 bits / (5000 x 10⁻³ sec)
= 100 bits/sec

Throughput Of Slotted Aloha-

- Throughput of slotted aloha = Efficiency x Bandwidth
= 0.368 x 100 Kbps
= 36.8 Kbps

Total Number Of Stations-

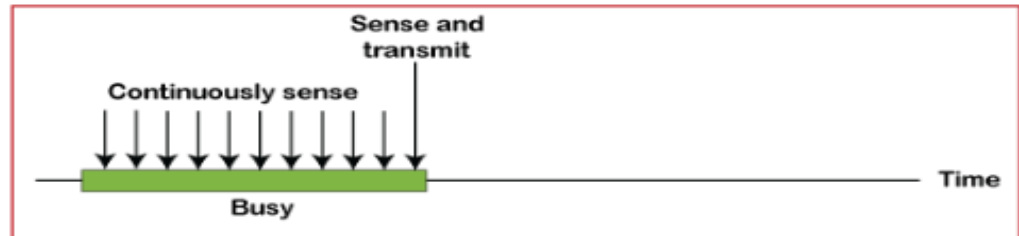
- Throughput of slotted aloha = Total number of stations x Throughput of each station
- Substituting the values, we get - 36.8 Kbps = N x 100 bits/sec
 $\therefore N = 368$ stations

CSMA (Carrier Sense Multiple Access)

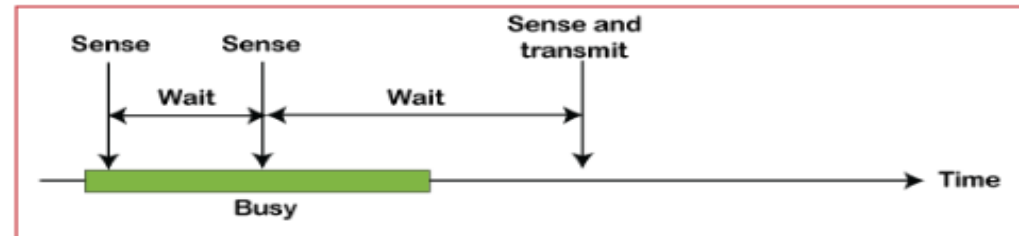
- It is a **carrier sense multiple access** based on media access protocol to sense the traffic on a channel (idle or busy) before transmitting the data.
- It means that if the channel is idle, the station can send data to the channel.
- Otherwise, it must wait until the channel becomes idle.
- Hence, it reduces the chances of a collision on a transmission medium.

CSMA Access Modes

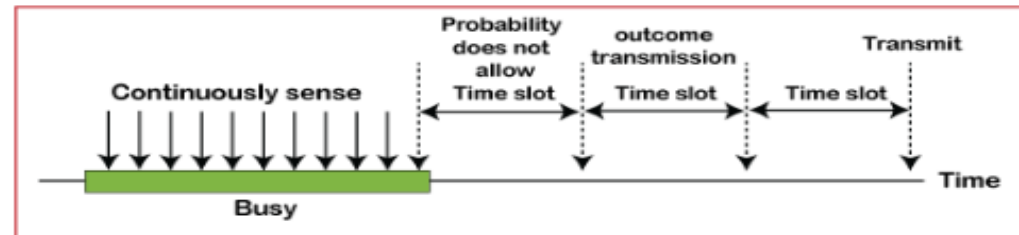
- 1-persistent
- Non-persistent
- p-persistent



a. 1-persistent



b. Nonpersistent



c. p-persistent

1-persistent

- In the 1-Persistent mode of CSMA that defines each node, first sense the shared channel and if the channel is idle, it immediately sends the data.
- Else it must wait and keep track of the status of the channel to be idle and broadcast the frame unconditionally as soon as the channel is idle.

Non-persistent

- It is the access mode of CSMA that defines before transmitting the data, each node must sense the channel, and if the channel is idle, it immediately sends the data.
- Otherwise, the station must wait for a random time (not continuously), and when the channel is found to be idle, it transmits the frames.

p-persistent

- It is the combination of 1-Persistent and Non-persistent modes.
- The P-Persistent mode defines that each node senses the channel, and if the channel is idle, it **sends a frame with a P probability**.
- If the **data is not transmitted, it waits for a ($q = 1-p$ probability) random time** and resumes the frame with the next time slot.

1-Persistent CSMA — Numerical

- **Question:**

- A CSMA network operates at a data rate of **10 Mbps**. The propagation delay between two stations is **25 μs** , and the transmission time of a frame is **200 μs** . A station uses **1-persistent CSMA**.

1. Calculate the propagation-to-transmission ratio a .
2. Explain what the station does when the channel is found **busy**.
3. If two stations sense the channel idle at the same instant and both start transmitting, explain what happens.
4. Calculate the approximate channel efficiency using the given transmission and propagation times, assuming no retransmission overhead

$$1. \ a = \frac{T_p}{T_t} = \frac{25 \mu s}{200 \mu s} = 0.125$$

2.

Busy $\rightarrow$ keep sensing

Idle $\rightarrow$ transmit immediately

3. Collision occurs.

$$4. \ \eta = \frac{1}{1+2a} = \frac{1}{1+0.25} = \frac{1}{1.25} = 0.8$$

Non-Persistent CSMA — Numerical

- **Question:**

Two stations use **non-persistent CSMA** to access a shared channel. The propagation delay is **20 μ s**, and the frame transmission time is **100 μ s**. The probability that the channel is busy when a station senses it is **0.4**. When the channel is busy, a station waits for a random time before sensing again.

1. Calculate $a = T_p / T_t$.
2. What is the probability that the channel is found idle on a sensing attempt?
3. If a station makes **5 independent sensing attempts**, calculate the probability that it finds the channel idle at least once.

1. $a = \frac{T_p}{T_t} = \frac{20 \mu s}{100 \mu s} = 0.02$

2. $P(\text{busy}) = 0.4; P(\text{idle}) = 1 - 0.4 = 0.6$

3. $P(\text{idle at least once}) = 1 - P(\text{never idle for all 5 times}) = 1 - [P(\text{busy}) \text{ AND } P(\text{busy}) \text{ AND } P(\text{busy}) \text{ AND } P(\text{busy}) \text{ AND } P(\text{busy})] = 1 - 0.4^5 = 1 - 0.01024 = 0.98976;$

98.976%

p-Persistent CSMA — Numerical

- **Question:**

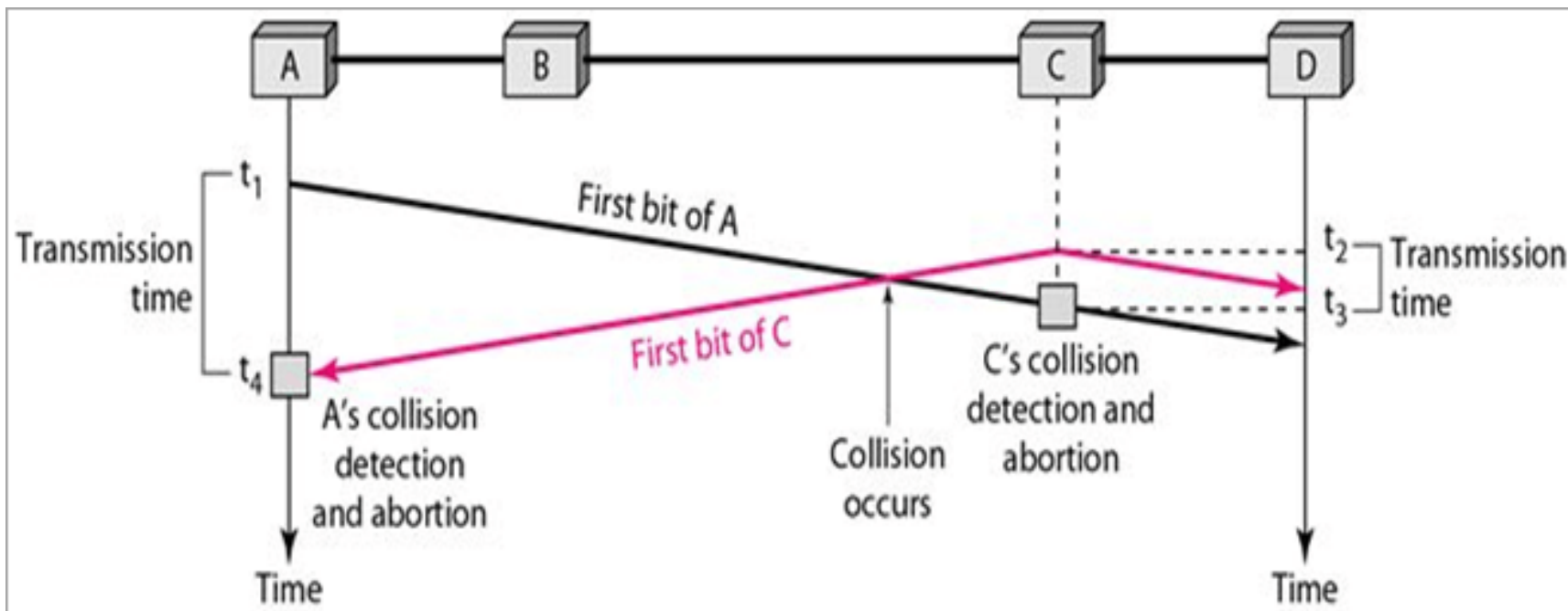
A slotted CSMA system uses **p-persistent CSMA** with $p=0.25$. Four stations are waiting to transmit, and the channel has just become idle.

1. What is the probability that a particular station transmits in the next slot?
2. What is the probability that a particular station **does not transmit**?
3. Calculate the probability that **exactly one** of the four stations transmits.
4. Calculate the probability that **none** of the four stations transmits.
5. Calculate the probability that **two or more** stations transmit simultaneously, causing a collision.

1. $P(\text{transmits in the next slot}) = 0.25$
2. $P(\text{does not transmit}) = 1 - 0.25 = 0.75$
3. Say, there are 4 stations: ABCD; Exactly 1 out of all transmits. A could only be transmitting/ B could only be transmitting/ C could only be transmitting/ D could only be transmitting.
 $(0.25)^1 (0.75)^3 \times 4 = 0.4219; 42.19\%$
4. $P(\text{none transmits}) = (0.75)^4 = 0.3164; 31.64\%$
5. $P(2 \text{ or more transmit}) = 1 - P(\text{no one transmits}) - P(1 \text{ transmits}) = 1 - 0.3164 - 0.4219 = 0.2617; 26.17\%$

CSMA/ CD

- It is a **carrier sense multiple access/ collision detection** network protocol to transmit data frames.
- The CSMA/CD protocol works with a medium access control layer.
- Therefore, it **first senses the shared channel before broadcasting the frames, and if the channel is idle, it transmits a frame to check whether the transmission was successful.**
- **If the frame is successfully received, the station sends another frame.**
- If any collision is detected in the CSMA/CD, **the station sends a jam/ stop signal to the shared channel to terminate data transmission.**
- After that, it **waits for a random time before sending a frame to a channel.**



- Station A,B,C, D are connected through Ethernet wire.
 - Any station can send its data packet for transmission after sensing the signal as idle.
 - Here the data packets are sent in bits which take time to travel.
 - Due to this, there are chances of a collision.
 - In the above diagram, at time t_1 station A starts transmitting the first bit of data after sensing the carrier as free.
 - At time t_2 , station C also senses the carrier as free and starts transmitting the data.
 - At t_3 , the collision occurs between bits sent by stations A and C.
- Thus, the transmission time for station C becomes $t_3 - t_2$.
- After the collision, the carrier will send back the colloidal signal to station A which will reach at time t_4 .
- This means, while sending the data, the collision can also be detected.

CSMA/ CA

- It is a **carrier sense multiple access/collision avoidance** network protocol for carrier transmission of data frames.
- It is a protocol that works with a medium access control layer.
- When a data frame is sent to a channel, it **receives an acknowledgment to check whether the channel is clear.**
- **If the station receives only a single (own) acknowledgments, that means the data frame has been successfully transmitted to the receiver.**
- **But if it gets two signals (its own and one more in which the collision of frames), a collision of the frame occurs in the shared channel.**
- **Detects the collision of the frame when a sender receives an acknowledgment signal.**

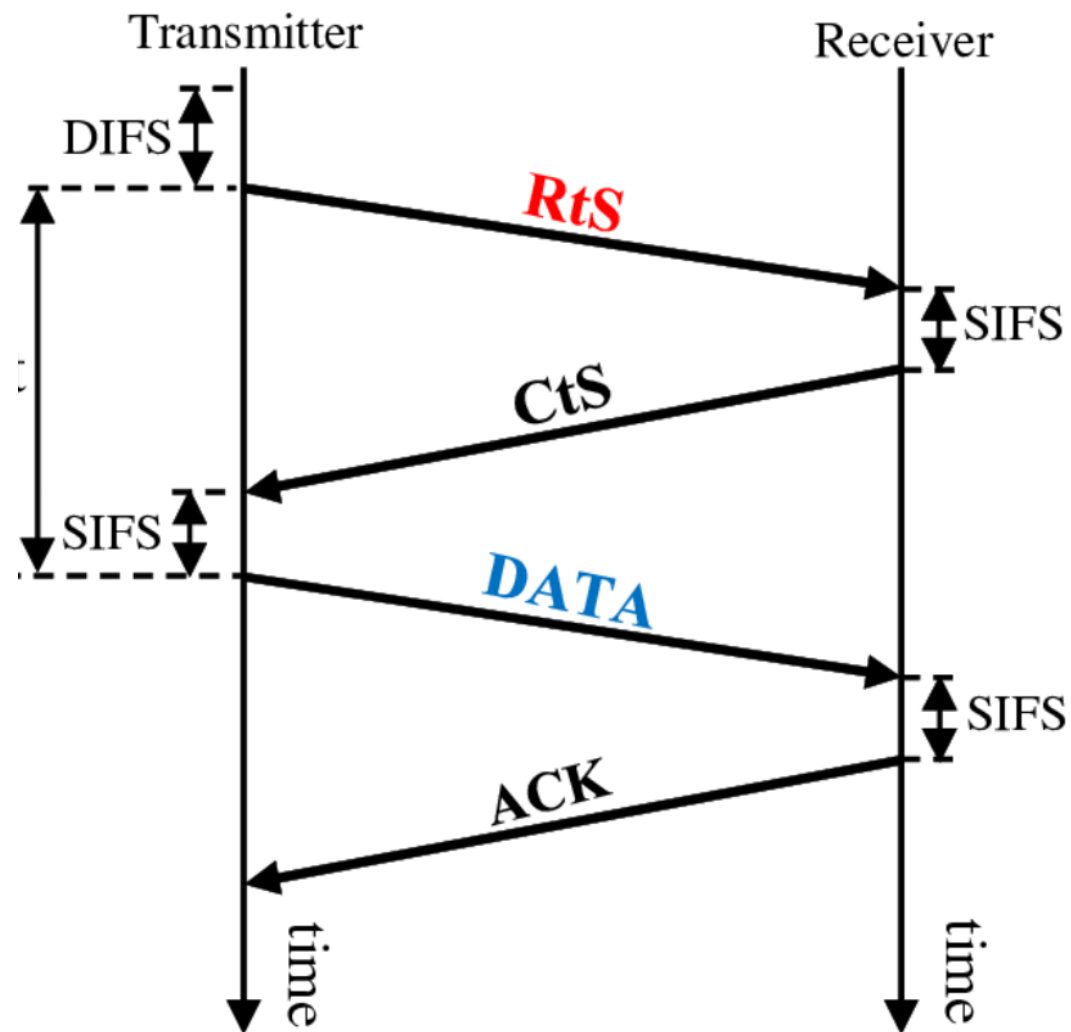
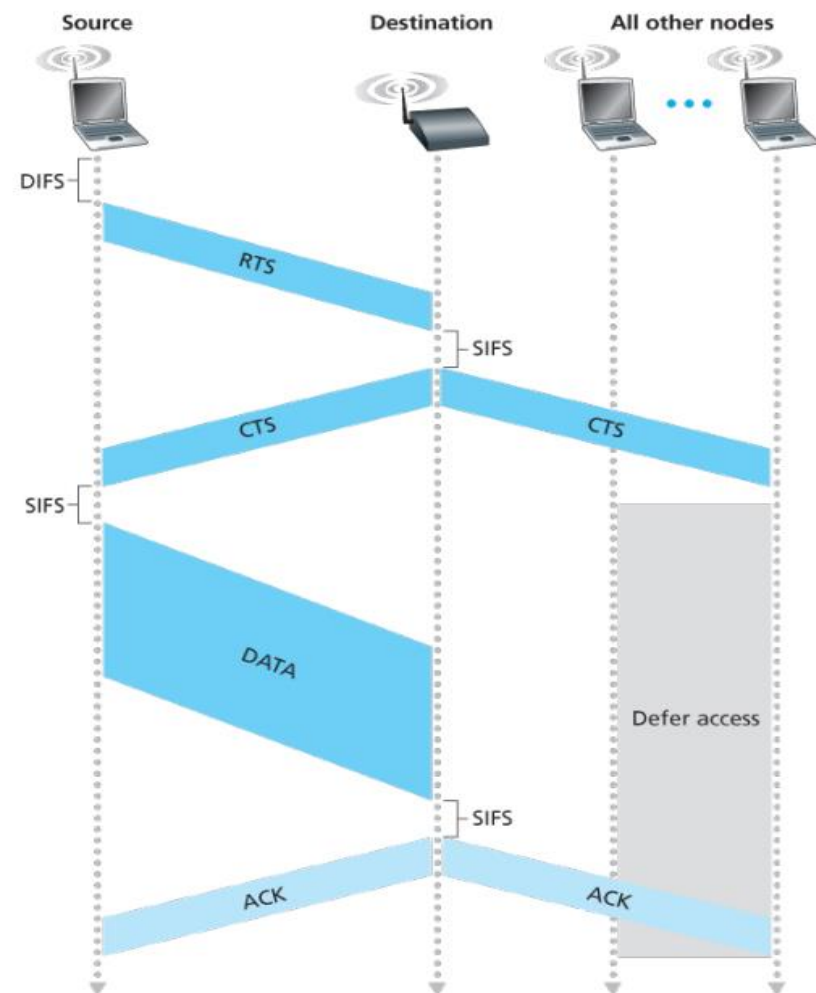


Figure 7.12 Collision avoidance using the RTS and CTS frames

- The transmitter initiates the CSMA/CA mechanism by sending a RTS control frame to ask the receiver if it is available.
- If it is the case, the receiver performs a rate adaptation algorithm by measuring the Signal to Noise Ratio (SNR) level of the received RTS frame to estimate the channel conditions.
- Then it replies with a Clear to Send (CTS) control frame to:
 - (i) inform other stations of its unavailability to receive information coming from other stations during a specified period;
 - (ii) suggest to the transmitter a transmission rate that the sender should use to transmit its physical packet.

By the way the receiver has a knowledge of the transmission rate corresponding to the value of the RATE subfield of the SIGNAL field.

THANK YOU !